Check for updates

Journal of Electrical and Computer Engineering

WILEY

REVIEW ARTICLE OPEN ACCESS

Large Language Models for Security Operations Centers: A Comprehensive Survey

Ali Habibzadeh | Farid Feyzi | Reza Ebrahimi Atani

Faculty of Engineering, University of Guilan, Rasht, Guilan, Iran

Correspondence: Farid Feyzi (feizi@guilan.ac.ir)

Received: 14 December 2025 | **Revised:** 25 April 2026 | **Accepted:** 18 May 2026

Academic Editor: Kusum Verma

Keywords: cybersecurity | generative AI | large language model | log analysis | network analysis | security operation centers | vulnerability analysis

ABSTRACT

Security operations centers (SOCs) face escalating challenges from alert fatigue and a critical skills gap, leading to delayed incident response times. Addressing these persistent issues calls for innovative automation and decision-support approaches. Large language models (LLMs) present a transformative opportunity to automate complex workflows and augment the capabilities of human analysts. This survey provides the first comprehensive and structured analysis of LLM integration into SOC operations. We systematically map LLM applications to the functions of the NIST Cybersecurity Framework (CSF) and use the MITRE ATT&CK framework as an analytical lens to evaluate their granular threat detection capabilities. By synthesizing findings from 216 studies, we provide a structured overview of current methodologies, identify key trade-offs between LLM architectures, and highlight significant gaps in research, particularly in the NIST "Recover" function. This survey offers researchers and SOC managers a clear research roadmap and actionable insights for leveraging LLMs to build more resilient and intelligent security operations.

1 | Introduction

Large language models (LLMs) represent a paradigm shift in artificial intelligence, emerging as large-scale, pretrained neural networks capable of comprehending and generating human-like language from massive textual corpora [1]. Their inherent versatility has established them as powerful tools for a wide array of downstream tasks, fueling a surge in research across diverse fields such as telecommunications [2], agriculture [3], transportation [4], and healthcare [5]. Despite this rapid expansion, the full potential of LLMs, particularly within technology-related disciplines, remains an active and promising frontier of exploration. One of these areas is cybersecurity.

The growing importance of cybersecurity is closely tied to the widespread integration of technology into modern society, which has exponentially expanded the attack surface [6]. The global cost of cybercrime was estimated to be around $8.4 trillion in 2022 and is predicted to reach $23.84 trillion annually by 2027 [7]. In response, security operations centers (SOCs) function as the central hub for an organization's cybersecurity operations [8]. However, their effectiveness is frequently hampered by operational challenges quantified in industry reports. According to the SANS 2025 SOC survey, the lack of skilled staff is their primary operational challenge, directly exacerbating alert fatigue and delayed detections [9]. This persistent shortage is driven by strong employer demand for security analysts, which the U.S. Bureau of Labor Statistics estimates will grow by more than 30% by 2030 [10]. Furthermore, the Ponemon Institute reports that the average time to identify and contain a breach is 258 days, a period during which attackers can freely operate within a network [11]. This context of operational strain is the critical problem space that LLMs promise to address.

Recent advances in LLM-based cybersecurity approaches provide an opportunity to re-examine the ways SOCs handle detection, analysis, and response compared to traditional methods. They

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

Copyright © 2026 Ali Habibzadeh et al. *Journal of Electrical and Computer Engineering* published by John Wiley & Sons Ltd.

offer unique features and capabilities; for example, in log analysis, they can automatically interpret heterogeneous and noisy logs that traditionally require manual triage. In vulnerability detection, they can reason over code semantics beyond the rigid rules of static or dynamic analysis. In phishing detection, they can assess linguistic patterns and contextual cues rather than relying solely on handcrafted signatures. In cyber threat intelligence (CTI) processing, they can rapidly summarize, correlate, and extract insights from diverse unstructured sources. Models adopting this paradigm aim to overcome long-standing limitations of traditional SOC workflows, which depend heavily on static rule sets, manually curated knowledge bases, and single-modality input constraints. Historically, tasks such as vulnerability detection or CTI consumption required substantial human effort, suffered from scalability issues, and were prone to errors such as high false-positive rates or missed insights. Consequently, these capabilities suggest that LLM-driven approaches have the potential to fundamentally reshape SOC workflows by improving automation, scalability, and the effectiveness of security operations.

While LLMs have been widely studied in cybersecurity, existing literature remains fragmented and fails to provide a holistic, operationally focused perspective for SOCs. Several surveys offer a broad overview of LLMs in security and privacy [12, 13], but they lack a structured analysis of LLM integration across the distinct phases of SOC operations. Other studies focus narrowly on specific tasks like vulnerability detection [14–16] or threat detection [17], but do not situate their findings within the broader context of a SOC's workflow and requirements. Furthermore, prior research on SOCs itself is outdated, having been published before the recent rise of generative AI [18, 19]. This indicates a clear gap in contemporary research for a systematic, up-to-date investigation of LLMs within the operational environment of SOCs.

To address this gap, this survey provides a structured overview of the academic literature on leveraging LLMs for SOC tasks. This workflow is demonstrated in Figure 1. Our work offers several unique contributions:

1. A foundational overview: We provide a clear and structured explanation of SOC operations, including analyst tiers, workflow phases, and the challenges they face in the modern threat landscape.
2. A structured analysis: We introduce a structured analysis that maps LLM applications to the authoritative NIST Cybersecurity Framework (CSF) and uses the MITRE ATT&CK framework as a granular lens to evaluate threat detection capabilities. This approach distills key architectural patterns and provides a synthesized perspective on the field's evolution.
3. A research roadmap: We highlight the challenges of integrating LLMs into real-world SOC operations and identify critical future research directions, providing a clear path forward for both academia and industry.

The rest of the survey is organized as follows: Section 2 introduces preliminary concepts and related work. Section 3 details our systematic methodology and analytical framework. Section 4 presents the most widely adopted LLMs reported in the literature. Sections 5, 6, and 7 present our analysis of LLM applications across the SOC workflow, organized by the detection, analysis, and response phases. Finally, Section 8 outlines key challenges and future research directions.

2 | Preliminaries and Related Studies

This section introduces the fundamental concepts of LLMs and SOCs, followed by a review of related survey articles, with a discussion of how this work distinguishes itself from them.

2.1 | LLMs

LLMs are transformer-based neural networks, pretrained on vast datasets, that excel at understanding and generating human language [20]. The functional diversity of LLMs stems from three primary architectural paradigms. Encoder-only models (e.g., BERT) are specialized for language understanding, creating rich representations of input data. Conversely, decoder-only models (e.g., GPT and LLaMA) are designed for autoregressive text generation, excelling in tasks requiring detailed, sequential output. Encoder–decoder models (e.g., T5) synthesize these capabilities, supporting both comprehension and generation, although the distinct strengths of the first two have established them as the most prevalent choices in research applications [21].

To fully leverage the capabilities of these diverse LLM architectures and tailor them for particular applications, several key methods are employed:

- Pretraining from scratch: This involves training models from the ground up on large, specialized corpora to enable them to acquire domain-specific knowledge. This process requires massive datasets, often comprising billions or trillions of tokens, and is computationally intensive. Such approaches typically incur substantial computational costs, including extensive GPU/TPU resources, long training times, and high energy consumption, which may limit their accessibility to well-resourced organizations.
- Fine-tuning: This technique involves updating the parameters of pretrained LLMs using smaller, task-relevant datasets to adapt them to specific downstream tasks. Fine-tuning strategies are fundamental for specializing LLMs, enabling them to reliably address the intricate and dynamic demands of various real-world applications. Compared to pretraining, fine-tuning is significantly less computationally demanding, although costs can still vary depending on model size, dataset scale, and the chosen adaptation strategy (e.g., full fine-tuning vs. parameter-efficient methods).
- Prompt engineering: This method focuses on designing effective prompts to guide LLMs toward the desired outputs. Recent studies have demonstrated notable performance differences between simple and advanced prompting strategies [22–25]. From a computational perspective, prompt engineering is relatively low-cost, as it does not require model retraining, although more complex prompting techniques (e.g., chain-of-thought) may increase inference-time computational overhead.
- Retrieval-augmented generation (RAG): This technique combines LLMs with external knowledge repositories. It helps them to overcome limitations like generating incorrect

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

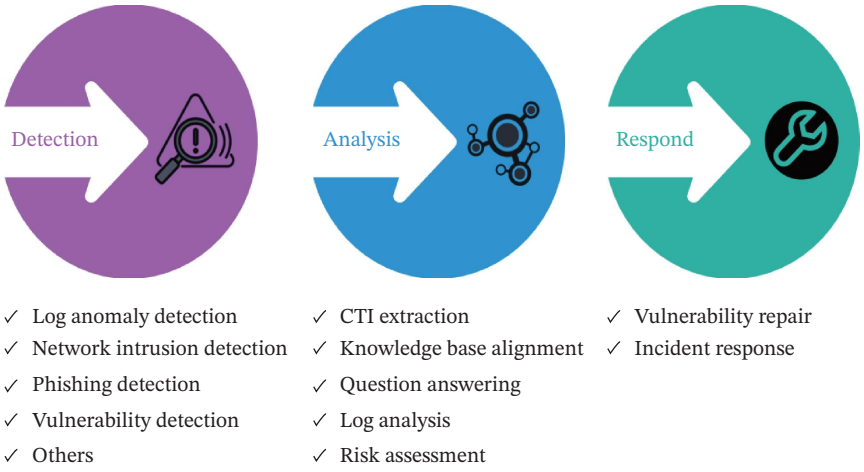


FIGURE 1 | Overall workflow of this survey.

information and lacking domain expertise. However, RAG introduces additional computational costs associated with indexing, retrieving, and integrating external data, as well as increased latency during inference due to the retrieval step.

- Agents: This approach involves equipping LLMs with external tools to expand their capabilities and enable them to perform complex tasks. LLM agents can serve as coordinators for intricate operations, and they have been shown to effectively manage multistep workflows, integrate diverse information sources, and improve overall task performance. Nevertheless, agent-based systems often incur higher computational costs due to iterative reasoning, multiple model calls, and interactions with external tools or APIs, which can significantly increase overall execution time and resource usage.

Leveraging these architectural types and adaptation methodologies, LLMs are increasingly being applied to solve complex problems across a multitude of industries. Their applications range from healthcare [26–29] to education [30–36] and software engineering [37–43]. In the further sections of this review, we examine the potential applications of LLMs in cybersecurity, with a particular focus on SOCs.

2.2 | SOCs

SOC is an organizational unit that integrates people, processes, and technologies to manage and strengthen an organization's overall security posture [44]. SOCs function as centralized hubs for cybersecurity, continuously monitoring, detecting, analyzing, and responding to security incidents. Their primary objectives are to protect sensitive data, ensure operational continuity, and safeguard the organization's reputation. Analysts within a SOC are typically structured into Tiers 1–4 according to their experience and responsibilities, as illustrated in Figure 2. Tier 1 analysts, often junior, focusing on initial alert triage, classification, and prioritization of security events to identify potential incidents. More complex or higher-severity incidents are escalated to Tier 2 analysts for in-depth investigation and remediation efforts, while the most experienced Tier 3 analysts typically perform proactive threat hunting and handle major incidents. Managing a SOC involves coordinating complex processes to mitigate risks, comply with regulatory requirements, and enhance security effectiveness, which is the duty of a Tier 4 analyst. This tiered structure ensures efficient handling of security events and proper escalation [45, 46].

Despite the existence of structured processes, modern SOCs encounter substantial operational challenges arising from the escalating volume, velocity, and variety of security data. These factors contribute to issues such as alert fatigue and an excessive number of false positives [47]. Coupled with a global skill shortage in cybersecurity and the monotonous, high-pressure nature of Tier 1 analyst tasks, this leads to analyst burnout and high turnover rates, compromising overall SOC effectiveness and increasing mean time to resolution for critical events [48]. Furthermore, the increasing sophistication and stealth of modern cyber threats, including advanced persistent threats (APTs) and zero-day exploits, often evade traditional rule-based detection systems, demanding more advanced analytical capabilities [49, 50]. Generative AI, particularly LLMs, are poised to mitigate these challenges by automating key aspects of the SOC workflow. These technologies can significantly improve the accuracy and efficiency of alert classification and prioritization, reducing false-positive rates and freeing human analysts from repetitive tasks to focus on complex investigations and threat hunting.

To systematically analyze the integration of LLMs into SOC operations, we ground our discussion in the widely adopted NIST CSF [51]. The NIST CSF provides a high-level, strategic framework for managing and reducing cybersecurity risk, organized into five core functions: Identify, Protect, Detect, Respond, and Recover. Our survey focuses on three core operational phases of a SOC and maps them directly to these functions, providing a structured lens for our analysis. The Detection phase corresponds to the NIST Identify and Detect functions, focusing on the development of activities to identify the occurrence of a cybersecurity event. The Analysis phase is a critical subcomponent that includes pieces from all three functions: Identify, Detect, and Response, involving triage, risk assessment, threat intelligence enrichment, and contextualization. Finally, the Response phase maps directly to the NIST "Respond" function, encompassing actions taken to contain, mitigate, and recover from a cybersecurity incident. While the NIST Protect and Recovery functions are foundational to a SOC's proactive posture, our survey focuses

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



FIGURE 2 | SOC analyst tiers. Analysts are typically organized into four tiers based on their expertise, responsibilities, and the complexity of alerts they handle.

on the reactive and analytical functions where LLMs are currently showing the most direct application.

2.3 | Analytical Framework: Mapping LLMs to NIST and ATT&CK

This survey systematically maps the application of LLMs across the SOC workflow to the functions of the NIST CSF framework and uses the MITRE ATT&CK framework to provide granular detail on threat detection capabilities. This dual-framework approach allows for a comprehensive analysis that spans both strategic management (NIST CSF) and operational execution (MITRE ATT&CK). To provide a high-level overview and a structural roadmap for the detailed analysis presented in Sections 5, 6, 7, we present the following mapping in Table 1. This table synthesizes the current state of research by illustrating which SOC tasks, aligned with NIST functions, are being enhanced by LLMs and which specific ATT&CK techniques they target. It serves as a visual guide to the reader, highlighting the areas of concentrated research and identifying potential gaps in the existing literature.

2.4 | Related Surveys

Since the emergence of LLMs, researchers have increasingly investigated their applications across diverse domains, including cybersecurity and software engineering. This has resulted in a growing body of survey articles that review, categorize, and compare existing studies in these fields. Table 2 presents a comparative overview of this survey in relation to other review studies. Several works specifically focus on providing a comprehensive overview of the literature related to either the offensive or defensive aspects of LLMs in cybersecurity. These studies provide a holistic perspective of the integration of LLMs into the cybersecurity domain [12, 13]. Although these types of surveys address various aspects of cybersecurity, their coverage often remains high-level and fragmented. They primarily mention the applications of LLMs but lack a structured and rigorous comparison of the reviewed methodologies, therefore limiting the ability to discern strengths and weaknesses across studies. As a result, common patterns, prevailing practices, and critical research gaps are not sufficiently highlighted in the existing literature.

A substantial body of research has also focused on more specific tasks and application domains. Notably, software vulnerability detection and repair stand out as a particularly active and well-reviewed area within the broader intersection of LLMs and cybersecurity, reflecting its critical importance in safeguarding software systems [14–16]. In addition, several systematic reviews offer broader perspectives on the use of LLMs across other aspects of cybersecurity, including network operations [52] and threat detection [17]. Although these literature reviews examine the application of LLMs and their methodologies in specific downstream tasks within cybersecurity, they often fail to provide a comprehensive perspective on the broader operational context and the distinct requirements of SOC environments. This broader scope means that these studies may not comprehensively reflect the nuanced integration and capabilities of LLMs within various SOC workflows. Consequently, the information provided often falls short of offering sufficiently specific and actionable insights for SOC managers to effectively guide strategic decisions and enhance team performance.

Finally, it is noteworthy that only a limited number of studies have been published recently on SOCs, despite their critical role as central points of defense against cyberattacks. Among the existing work, two studies [18, 19] explored the use of machine learning to enhance SOC operations. Nevertheless, both were published prior to 2021. This indicates a clear gap in contemporary research, particularly in the integration of emerging AI techniques such as LLMs within SOC environments. Addressing this gap is essential for advancing both the practical capabilities of SOCs and the theoretical understanding of intelligent cybersecurity operations. Consequently, more systematic and up-to-date investigations are required to guide future developments and inform best practices in this rapidly evolving field.

3 | Methodology

Our survey methodology follows the systematic approach proposed by Kitchenham's guidelines [53] for performing systematic literature reviews (SLRs) in software engineering. This approach involves three major activities: planning, conducting, and reporting the review. In the planning phase, we formulated the research questions that guided this study and developed the

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | Mapping LLM applications in SOCs to NIST CSF and MITRE ATT&CK.

SOC phase	NIST CSF	Example LLM application	Targeted MITRE ATT&CK technique(s)
Detection	Identify/detect	Log-based anomaly detection, phishing detection, network intrusion detection, vulnerability analysis	T1059.001 (PowerShell), T1003 (OS credential dumping), T1566.001 (Spearphishing attachment), T1566.002 (Spearphishing link), T1498 (network denial of service), T1071.001 (web protocols), T1190 (exploit public-facing application)
Analysis	Identify/detect/respond	Risk assessment, alert triage and prioritization, CTI summarization and synthesis, incident contextualization	T1190 (exploit public-facing app), T1598 (Spearphishing), T1055 (process injection), T1566 (phishing), T1059.001 (PowerShell), T1021.002 (SMB/windows admin shares)
Response	Respond	Automated containment, dynamic response orchestration	T1055 (process injection), T1561.001 (disk content wipe)
[Research gap]	Recover	Automated system restoration, postincident reporting	T1485 (data destruction), T1490 (inhibit system recovery)

search queries for identifying relevant publications. During the conducting phase, we searched across selected digital libraries and retrieved the initial set of studies. We then applied inclusion and exclusion criteria to refine this set to the final group of studies included in our analysis. To ensure relevance to our research questions, we screened the selected studies based on their titles, abstracts, and conclusions. Full-text reviews were conducted when necessary to resolve ambiguities and confirm eligibility. Finally, in the reporting phase, we synthesized and organized the extracted data to systematically highlight key findings.

3.1 | Research Questions

To understand the current state of LLM-based models in SOC-related tasks, it is necessary to create appropriate and effective research questions. Through this SLR, five research questions are defined as follows, which are answered in Sections 4 to 8.

- RQ1: What LLMs have been utilized?
- RQ2: How are LLMs adapted for the detection phase of SOCs?
- RQ3: How are LLMs adapted for the analysis phase of SOCs?
- RQ4: How are LLMs adapted for the response phase of SOCs?
- RQ5: What are the challenges and future research directions for applying LLMs in the SOC domain?

Within the research questions, RQ1 seeks to identify specific patterns between the reviewed papers and the LLMs employed, enabling researchers to make more informed choices regarding the LLM most appropriate for their downstream tasks. Moreover, RQ2 to RQ4 focus on methodologies for employing LLMs and comparing their performance across different phases of SOC operations. Lastly, RQ5 outlines the challenges in this research area and identifies potential future directions for the field.

To address RQ2–RQ4 and provide a clearer understanding of how generative AI is being integrated into SOC workflows, this survey adopts the Technology Readiness Level (TRL) framework. TRL is a rigorous maturity scale that was originally developed by NASA [54] and later standardized internationally, including through ISO 16290 [55]. The framework consists of nine levels that track the progression of a technology from the observation of basic principles to the deployment of fully operational systems. These levels start with basic principles observed and end with actual system proven in an operational environment. A technology's placement within a specific TRL level is determined by the extent to which its underlying concepts have been validated, demonstrated, and integrated under progressively realistic conditions. In this survey, we use the TRL framework to systematically assess the maturity of LLM applications within SOC contexts based on evidence gathered from the reviewed literature. However, for the purposes of this study, these nine levels are consolidated into four broader categories to facilitate comparison across different SOC tasks:

1. Emerging stage: Concept development and proof of concept
2. Performance validation: Laboratory and pilot validation

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 | Comparison of related surveys on LLMs and cybersecurity.

Reference	Primary focus	Scope	Key gap addressed by our work
[12, 13]	LLMs in security/privacy	Broad	Lacks focus on SOC operational workflow
[14–16]	LLMs for vulnerability detection	Specific task	Narrow focus; not a comprehensive SOC view
[17]	LLMs for threat detection	Specific task	Lacks analysis of LLM integration across SOC phases
[18, 19]	ML in SOCs	Outdated (pre-LLM)	Does not cover modern generative AI techniques

3. Operational adaptation: System prototype demonstrated in operational environments
4. Full implementation: Complete, qualified, and proven systems ready for deployment

This consolidation enables a clearer, more coherent assessment of technological progress and highlights maturity differences across the diverse application areas examined.

3.2 | Search Strategy

A search strategy is a structured approach to identifying relevant studies in order to provide a comprehensive and unbiased overview of the literature. This study primarily utilized five major scientific databases: ScienceDirect, SpringerLink, IEEE Xplore, ACM Digital Library, and arXiv. To supplement these sources, Google Scholar was employed to capture additional relevant publications that may not have been indexed in the primary databases. We conducted the initial search using broad queries to capture a comprehensive set of papers in the field, followed by a manual screening process to eliminate irrelevant studies. The initial search queries focused on keywords related to LLMs, cybersecurity, and SOC. We employed the following search string to identify all primary studies relevant to the topic: (Large Language Model OR LLM OR Generative AI OR BERT OR GPT OR LLama) AND (Cybersecurity OR Security Operation Center OR SOC OR Log OR Vulnerability OR Network OR Cyber Threat OR Cyber Threat Intelligence).

3.3 | Study Selection

Study selection involves applying predefined inclusion and exclusion criteria to determine which papers are relevant for the review. This process ensures that only studies directly addressing the research questions are analyzed, improving the rigor and focus of the survey. We restricted our search to papers published between 2021 and 2025, ensuring the study captures the latest advancements in the field. Additionally, studies were included only if they directly employed LLMs in their architecture and their downstream task aligns with SOCs. We excluded duplicate studies, non-English publications, and documents that were not full-length papers, including books, short papers, and tutorials. In the next step, each paper was screened based on its abstract and conclusion to evaluate its relevance, novelty, and currency. When ambiguities remained after this step, the full text was reviewed to determine eligibility. Through this process, we ensured that only relevant, novel, and methodologically significant studies were included in the final review.

3.4 | Data Extraction

Data extraction and categorization systematically collect and organize key information from selected studies to enable structured analysis and meaningful comparisons across the literature. Following the above processes, we identified 216 articles, and their distribution by published year is illustrated in Figure 3. The chart indicates a sharp increase in the introduction of LLM-based models within SOC-related domains, a trend that is likely to continue as LLMs advance further. The selected papers were categorized according to the workflow of SOCs, allowing for a structured analysis of the role of LLMs across each phase. The distribution of papers reveals a clear decline in the number of studies from Detection to Response. This suggests a significant research gap in the Response phase, likely due to the higher operational risk, stricter reliability requirements, limited availability of real-world datasets, and the openness of the operational space in which a large number of potential response actions and event outcomes must be considered.

4 | RQ1: What LLMs and Methods Have Been Utilized?

Figure 4 illustrates the most frequently adopted LLMs and methods among the studies reviewed in this paper. The dominance of BERT-based models in the Detection phase is a direct consequence of their superior ability to understand and classify. For tasks like log anomaly detection, the goal is to map an input log sequence to a class (e.g., “normal” or “anomaly”). The bi-directional attention mechanism in BERT is perfectly suited for this, as it can build a deep contextual understanding of the entire log entry before making a classification decision. Moreover, the widespread use of fine-tuning stems from the need to adapt pretrained language models to cybersecurity tasks, which rely on specialized data formats such as logs, network traffic, and vulnerability codes that differ from standard natural language data.

Conversely, the rise of GPT-based models in the Analysis and Response phases is driven by the generative nature of the tasks. An analyst does not just need a classification for an alert; they need an explanation, a summary of related threat intelligence, or a draft containment script. These are inherently generative tasks that fall squarely into the domain of decoder-only models. For instance, an LLM like GPT-4 can ingest a complex alert and generate a human-readable summary: “This alert indicates a potential PowerShell-based lateral movement attack (ATT&CK T1059.001). The source IP is associated with the APT28 group. Recommended action is to isolate the endpoint and capture a memory dump.” This level of generative synthesis is beyond the capabilities of an encoder-only model. Therefore, the choice of architecture is a critical first step in designing an LLM-powered SOC tool. It dictates the model’s fundamental capabilities, its computational requirements, and ultimately, its suitability for solving a specific problem within the high-stakes environment of a security operations center. Similarly, the capabilities of prompt engineering, including the flexibility and efficiency of zero-shot

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

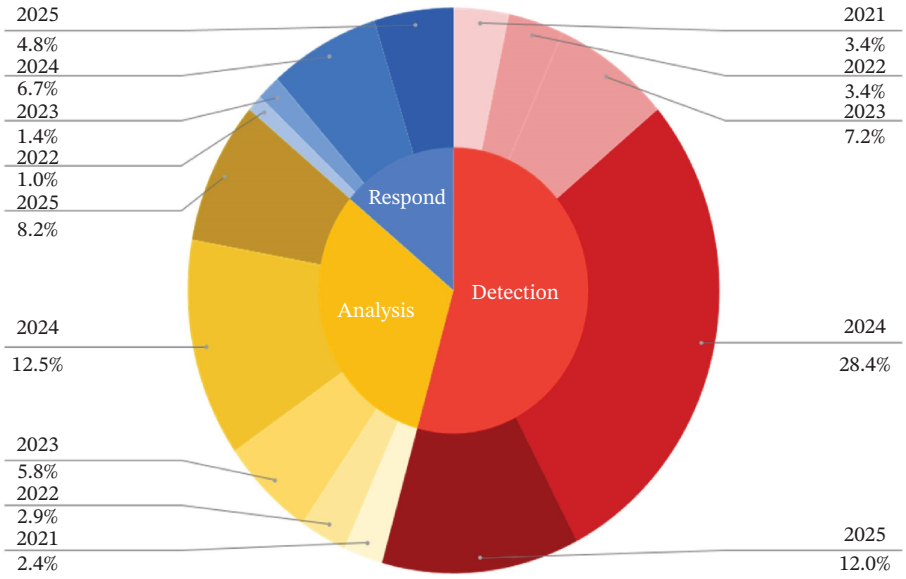


FIGURE 3 | Distribution of reviewed papers by SOC workflow phase and publication year.

and few-shot learning, make it an ideal approach for more general tasks during analysis and response phases. Ongoing research and development are essential to refine these techniques, address inherent limitations like biases and interpretability, and ensure the robust and reliable deployment of LLM-powered solutions in sensitive security environments.

Answer to RQ1: BERT and GPT families are the most widely used models in SOC-related studies, primarily due to their trainability and in-context learning capabilities, respectively. Among the presented methods, fine-tuning and prompt engineering are the most commonly adopted approaches in the literature. This observation can be attributed to the need for domain-specific knowledge in certain tasks and the efficiency of zero-shot or few-shot task adaptation.

5 | RQ2: How Are LLMs Adapted for the Detection Phase of SOCs?

The Detection phase is a central stage in the SOC workflow, corresponding directly to the NIST Identify and Detect functions. This phase is dedicated to identifying potential security incidents through continuous monitoring of IT infrastructure and security controls. The primary goal is to discover anomalous patterns and potential threats that may be part of an unfolding attack. LLMs are increasingly being leveraged to enhance this function by

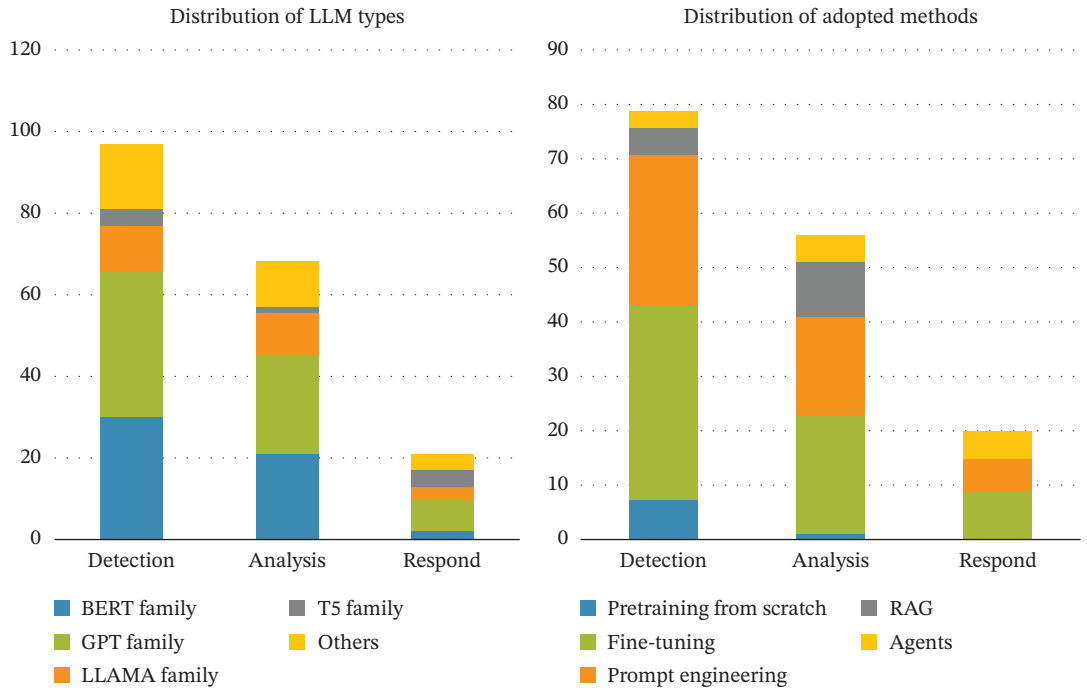


FIGURE 4 | Distribution of reviewed papers by SOC workflow, categorized by the LLM type and adoption method.

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

automating the analysis of vast and diverse data sources, such as system logs, network traffic, and communications. In the remainder of this section, we examine the application of LLMs in the Detection phase by demonstrating their effectiveness in identifying specific ATT&CK techniques.

5.1 | Log Anomaly Detection

System logs serve as a primary source of information for monitoring and maintaining the security of computer systems. Their proper interpretation is essential, as SIEM tools rely on logs to observe system behavior and detect security threats. LLMs have advanced this domain by enabling the comprehension of the semantic and temporal structure of logs, surpassing traditional approaches that depend on manual feature engineering and syntactic parsing. A substantial body of research now focuses on employing LLMs to detect malicious activities within system logs, which directly corresponds to the identification of several ATT&CK techniques. For example, several models can be trained to recognize anomalous command-line executions, thereby enhancing the detection of MITRE ATT&CK technique T1059.001 (Command and Scripting Interpreter: PowerShell), a common vector for privilege escalation and lateral movement. They also can focus on identifying patterns associated with T1003 (OS Credential Dumping) by flagging unusual access attempts to sensitive system files like the Security Account Manager database. LLM-based approaches offer a significant advantage over traditional signature-based methods by leveraging contextual command understanding to accurately distinguish legitimate administrative scripts from malicious activity.

The LLM-based models proposed in this field can be divided into two categories: Training-based methods and prompt engineering-based methods. Reconstruction-based models place within training-based methods, which involves training an LLM only on normal data. These models can effectively predict the normal behavior of the system and detect anomalies by analyzing the differences between the predicted logs and those generated by the system. Specifically, most models [56–58] map log sentences to template IDs referred to as log key and detects anomalies by comparing the actual log key of randomly masked tokens with top k predictions at inference time. The value of the hyperparameter K could have a large impact on the performance of these kinds of models and determines the strictness. A smaller K usually leads to high recall and low precision, and a larger K results in high precision and low recall.

The primary distinction of LogFiT [59] compared to prior models lies in its use of raw log sentences rather than the log keys, coupled with its exploitation of the pretrained knowledge embedded within LLMs. Approaches relying on log keys exhibit semantic limitations, as they fail to capture the nuanced meanings within log sentences. In addition, they struggle to accommodate variability in log content and alterations in the lexical structure of logs over time. Unseen or slightly modified log sentences often fail to align with existing templates, which can lead to degraded performance [60]. Reconstruction-based models have the benefit of not requiring any anomaly data, but they also have the drawback of requiring constant training on normal data to adapt to new emerging patterns.

Binary classification models represent another strategy, which requires training. These models consist of two part: The first part involves utilizing a LLM for extracting semantic vectors from log messages and second part involves a deep learning model for binary classification, which could be MLP [61], custom Siamese network [62], or another language model [63]. Although binary classification models demonstrate strong performance, their large number of parameters results in substantial computational overhead and prolonged processing time, making them impractical for most operational deployments. Additionally, challenges such as training data imbalance and the limited availability of anomaly samples have reduced researchers' interest in this approach. Several recent studies [64–66] have also demonstrated that training a language model on multiple related tasks, rather than exclusively on a single task, can enhance its performance on each individual task.

Prompt engineering methods guide LLMs to detect log anomalies by crafting detailed prompts. LogGPT [67] and LogPrompt [68] are both exploring the capacity of GPT-3.5 on log analyzing tasks by examining different prompt engineering techniques such as self-prompt, chain-of-thought, and few-shot learning. Although these models generally underperform compared to training-based approaches, they offer the advantages of not requiring training data and exhibiting strong generalizability. They also demonstrate acceptable interpretability, providing reports and preventive suggestions for detected anomalies, which can aid administrators in troubleshooting. Another study [69] proposed a multiagent framework designed for log-based insider threat detection (ITD). The framework comprises three collaborative agents working together to tackle the complexities of ITD. Furthermore, to address faithfulness hallucination [70] and increase accuracy, Audit-LLM incorporates a pairwise evidence-based multiagent debate mechanism, where two independent executors iteratively refine their conclusions through reasoning exchange to reach a more reliable consensus.

RAGLog [71] and RAPID [72] are two retrieval-based models that involves finding known normal log sequences corresponding to new test logs and judging them based on the comparison between them. These models aim to address the limitations of existing LLM-based methods, including constraints related to token capacity, memory retention, hallucination, and real-time applicability. The innovation of RAPID lies in its focus on token-level information, enabling the detection of subtle anomalies that are often overlooked by sequence-level methods. The method's capability to function without prior training, combined with its efficiency and robustness in handling unseen logs, makes it particularly well-suited for real-time anomaly detection in dynamic environments.

The abundance of research published in this area, coupled with the consistency of results reported across diverse datasets and experimental conditions, has established a strong foundation for this line of work and positions LLM-based approaches within the performance validation stage of the TRL framework. Despite this widespread adoption, several challenges need to be addressed before real-world deployment. These obstacles involve context length limitations that restrict the processing of extensive log files, contradictory outputs, and high computational costs that impede real-time deployment. Additionally, LLMs often struggle with domain-specific terminology found in system logs. Future research aims to mitigate these limitations by enhancing result consistency and advancing real-time response capabilities

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

through the development of lightweight models capable of efficiently handling large-scale log analysis [71]. Table 3 presents the log anomaly detection models that incorporate LLMs in their architecture. A parser is a preprocessing component that transforms unstructured raw log messages into structured templates, enabling downstream models to effectively identify anomalous patterns. Among the models that include such a parser, Drain [77] is the most widely adopted choice across the reviewed literature.

Loghub [73] is a widely utilized benchmark for system log analysis, supporting a broad range of tasks including anomaly detection, log parsing, failure prediction, log compression, and log summarization. The repository contains approximately 85 GB of raw log data. This benchmark, particularly its Loghub-2k version, comprises 16 diverse log datasets including those from distributed systems, supercomputers, operating systems, mobile systems, server applications, and standalone software. For evaluation, 2000 log messages from each system were manually labeled with ground-truth templates, and some datasets are also manually labeled for anomalies, making them suitable for anomaly detection tasks. HDFS, BGL, and Thunderbird are among the most frequently used datasets from Loghub for anomaly detection.

5.2 | Network Intrusion Detection

Network intrusion detection systems (NIDSs) are crucial for monitoring network traffic to identify malicious activities. LLM-based systems are enabling a more proactive security posture by identifying subtle signs of network intrusions before they fully manifest. Researchers in this area have successfully leveraged the capabilities of LLMs to defend against a range of network-based ATT&CK techniques. For instance, one study could leverage LLMs' contextual understanding to improve the detection of DDoS attacks, directly targeting MITRE ATT&CK technique T1498 (network denial of service). Moreover, other works can focus on analyzing network traffic to identify command-and-control (C2) communications, which aligns with T1071 (application layer protocol) and T1095 (nonapplication layer protocol). Nonetheless, these models also present challenges, such as limited understanding of network-specific protocols [78].

As shown in Table 4, the direct application of prompt engineering methods within the field of network intrusion detection remains comparatively uncommon. This limited adoption is primarily attributed to the substantial knowledge gap between the natural language understanding capabilities of LLMs and the low-level, structured, and often protocol-specific nature of network traffic data. One way to bridge this gap is translating raw network data into semantically meaningful representations that LLMs can effectively interpret [79, 87]. In this context, a study [79] proposes a novel technique called privacy-preserving fixed-length encoding (PPFLE), which transforms numerical and categorical data into a textual representation. PPFLE concatenates column names with their respective values and then hashes them, effectively creating a new language that BERT can understand, while also preserving data privacy by obscuring sensitive information.

Existing research [85] investigates the application of LLMs on understanding nonlanguage network data for detecting unknown malicious flows, specifically focusing on carpet bombing

TABLE 3 | Overview of LLM-based Log anomaly detection models.

Reference	Name	Methods	Training data	LLM	Parser?	Datasets	*F1*
[56]	LogBERT (2021)	Pretraining from scratch	Normal	BERT	✓	[73]	0.823, 0.908, 0.966
[61]	BERT-Log (2022)	Pretraining from scratch, Fine-tuning	Normal, abnormal	BERT	✓	[73]	0.996, 0.994
[57]	LogGPT (2023)	Pretraining from scratch, Fine-tuning	Normal	GPT-2	✓	[73]	0.901, 0.977, 0.986
[58]	LAno-BERT (2023)	Pretraining from scratch	Normal	BERT	✗	[73]	0.964, 0.875, 0.999
[67]	LogGPT (2023)	Prompt engineering	—	GPT-3.5	✓	[73]	0.625, 0.740
[59]	LogFiT (2024)	Fine-tuning	Normal	RoBERTa, Longformer	✗	[73]	0.950, 0.899, 0.939
[62]	SaRLog (2024)	Fine-tuning	Normal, abnormal	Secure-BERT	✗	[73]	0.988, 0.999
[71]	RAGLog (2024)	RAG	Normal	GPT-3.5	✗	[73]	0.890
[68]	LogPrompt (2024)	Prompt engineering	—	GPT-3.5	✓	[73]	0.384, 0.450
[72]	Rapid (2024)	RAG	Normal	BERT	✗	[73]	0.924, 0.999, 0.997
[69]	Audit-LLM (2024)	LLM-based agents	—	GPT-3.5	✗	[74, 75]	(Acc) 0.961, 0.959, 0.931
[64]	LogLM (2025)	Instruction tuning	Normal, abnormal	LLaMA-2	✓	[73]	0.811, 0.584
[65]	LUK (2025)	Multiexpert collaboration, fine-tuning	Normal, abnormal	GPT-4o, LLama3, BERT	✓	[73]	99.93
[63]	LogLLM (2025)	Fine-tuning	Normal, abnormal	BERT, Llama-3	✗	[73, 76]	0.997, 0.916, 0.958, 0.966
[66]	LLM-LADE (2025)	Fine-tuning	Normal, abnormal	LLaMA3	✓	[73]	0.990, 0.989, 0.997

Abbreviation: Acc, Accuracy.

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 4 | Overview of network Intrusion detection models.

Reference	Name	Method	LLM	Datasets	*F*-1	Key features
[79]	SecurityBERT (2024)	Pretraining from scratch, Fine-tuning	BERT	[80]	0.840	Leveraging a privacy-preserving encoding technique, generalizability is not verified
[81]	BERTIDS (2024)	Fine-tuning	BERT	[82]	98.090	Poor generalization
[83]	— (2024)	Fine-tuning	GPT-2, BERT	[84]	0.930	Predictive intrusion detection, efficiency is not verified
[85]	DoLLM (2024)	Representation learning	Llama2	[86]	0.938	Carpet bombing DDoS detection model, significant computational overhead, inference latency
[87]	IOV-BERT-IDS (2024)	Pretraining from scratch, Fine-tuning	BERT	[88–90]	1.000, 1.000, 0.999, 0.998	Vehicular network intrusion detection model, not efficient enough due to limited vehicular resources
[91]	IDS-agent (2024)	LLM-based agent	GPT-4o	[84, 92]	0.970, 0.750	Capable of explicit knowledge integration and explanation, efficiency is not verified
[93]	— (2024)	Prompt engineering	GPT-4	[94]	over 0.950	Evaluates different in-context learning techniques
[95]	BARTPredict (2025)	Fine-tuning	BART, BERT	[84]	0.982	Predictive intrusion detection, efficiency is not verified
[96]	RLFE-IDS (2025)	RAG	Chat-GLM3	[84, 86, 88, 97]	(Acc) 0.980	Knowledge retrieval, simplified deployment
[98]	FedLLMGuard (2025)	Prompt-tuning	Tiny-BERT	[86, 99, 100]	0.994, 0.997, 0.989	FL-LLM Integration

Abbreviation: Acc, Accuracy.

DDoS attacks. Carpet bombing is a recent and growing threat characterized by low-rate, multivector attacks targeting numerous victim IPs across subnets, leading to access link congestion and service disruption. In their study, they leverage LLMs' contextual understanding to extract powerful flow representations, moving beyond traditional feature engineering to improve DDoS detection performance.

There are some research endeavors actively focused on making LLM-based NIDS suitable for practical network operations by mitigating their inherent high computational requirements and inference latency, often through the development of lightweight architectures, prompt optimization, and their integration into hybrid frameworks that concurrently enhance explainability and adaptability to evolving threats [96, 101]. In a recent study [96], they propose the RLFE-IDS framework to overcome the significant difficulties and computational demands associated with directly deploying and fine-tuning LLMs for intrusion detection. This framework comprises RAG and a custom embedding model called FE-Net. FE-Net is specifically designed to transform network traffic data into feature vectors that preserve crucial correlations and are suitable for storage in this knowledge database, enabling the retrieval of relevant information for the LLM.

Another common and emerging concept in NIDS is federated learning, which addresses critical limitations of traditional centralized machine learning methods in network environments. FL is an avant-garde machine learning technique that enables distributed agents to collaboratively train a shared model without exposing sensitive local data, thereby mitigating issues such as high latency, significant communication overheads, and privacy risks associated with centralized data aggregation. This approach enhances confidentiality, scalability, and fault tolerance in NIDS deployments, particularly in resource-constrained IoT environments [102, 103]. Recently, researchers have integrated FL with LLMs to further enhance anomaly detection, addressing the limitation of the existing FL-based systems that rely heavily on numerical features and thus miss semantic and contextual nuances of evolving threats [98, 104]. As an example, FedLLM-Guard [98] is a framework designed for real-time, privacy-preserving, and adaptive anomaly detection in 5G networks. This model deploys lightweight LLMs at the client level to analyze text-based traffic logs and generate semantic embeddings and utilizes soft prompt tuning to reduce computational and communication overhead during the federated optimization loop.

While previous studies effectively detect known attacks and stop ongoing threats, they frequently fall short in predicting multistage attacks and preventing catastrophic damages, highlighting the pressing requirement for innovative approaches that enhance proactive threat prediction capabilities. Predictive intrusion detection represents a critical evolution in cybersecurity, moving beyond the traditional reactive nature of NIDSs. The core objective of this forward-looking approach is to anticipate and preemptively mitigate malicious activities before they can cause damage, thus significantly reducing the likelihood and impact of successful intrusions [83, 95]. One common approach in this context is to employ a multimodel framework for proactive threat prediction. This strategy typically involves a generative model to anticipate subsequent network events, a discriminative model to

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

validate the legitimacy of these predictions, and a final classifier to distinguish between benign and malicious activity. LLMs can also generate detailed, human-readable explanations for NID classifications, improving security professionals' understanding and trust in system predictions. By incorporating relevant background knowledge and summarizing complex network events, they support threat response and help interpret intricate attack patterns, thereby enhancing the interpretability and transparency of NIDS [78, 91].

The nonlinguistic nature of network intrusion detection, combined with the limited availability of systematically validated evaluations and the highly diverse research landscape in this domain, indicates that LLM-based approaches remain in the early stages of technological readiness within the TRL framework. This made many studies advocate for leveraging them as complementary agents within IDS frameworks to improve explainability and assist human interpretation, a strategy motivated by their substantially higher computational costs and inference times compared with traditional machine learning models [105–107].

Labeled datasets are necessary in this domain to train and evaluate machine learning-based NIDS as they allow models to learn from both normal and malicious network traffic patterns. However, many traditional datasets have limitations such as being outdated, lacking traffic diversity, containing anonymized packet information, or not accurately reflecting contemporary real-world network environments [108]. To address these shortcomings, new and comprehensive datasets have been developed, including CICIDS2017 [88] containing seven attack types, TON_IoT [109] with nine attack types, ACI-IoT-2023 [92] comprising four attack types, CICIoT2023 [84] featuring seven attack types, UNSW-NB15 [100] providing 20 attack types, and Edge-IIoTset [80] offering five attack types. These datasets supply diverse and realistic attack scenarios to support robust NIDS research and deployment.

5.3 | Phishing Detection

Cybercriminals favor attacks that maximize damage while minimizing effort, with phishing representing one of the most prevalent forms of social engineering that satisfies this criterion. Phishing involves the use of digital communications to deceive individuals into revealing sensitive data. It could cause substantial financial loss and setbacks for individuals, businesses, and economies globally. Phishing attacks have shown a marked rise in both frequency and complexity, as evidenced by a 17.3% increase in phishing emails recorded between September 15, 2024, and February 14, 2025. A significant proportion of these attacks, accounting for 82.6%, incorporated AI, representing a 53.5% year-over-year increase in the adoption of AI-driven techniques within phishing campaigns. These findings underscore the necessity of adopting more advanced strategies to effectively counter such threats [110]. This attack corresponds to Technique T1566 (Phishing) in the MITRE ATT&CK framework, which encompasses four subtechniques: T1566.001 (Spearphishing Attachment), where malicious files are delivered via email; T1566.002 (Spearphishing Link), which uses deceptive URLs in messages; T1566.003 (Spearphishing via Service), which leverages third-party communication platforms; and T1566.004 (Spearphishing Voice), also known as vishing. A comprehensive understanding of these subtechniques is essential for developing robust detection and mitigation strategies.

LLMs represent a transformative advancement in phishing detection by leveraging sophisticated language understanding to analyze diverse inputs such as emails, URLs, and HTML content. As illustrated in Table 5, the primary LLM-based phishing detection approaches can be categorized into email-based, URL-based, brand-based, and HTML-based methods. In the email-based approach, LLMs analyze the content and social engineering tactics of an email, effectively identifying T1566.001 (Spearphishing Attachment). This approach treats this task as a text classification problem. Some studies have evaluated the performance of pretrained knowledge in different language models using prompt engineering methods [117, 122], while others have fine-tuned specific models on these texts [111, 118], and still others have employed more innovative approaches [133]. In one such study [133], the authors introduce prompted contextual document vectors, a technique where an ensemble of LLMs is probed with human-crafted questions to quantify social engineering tactics, with the resulting probabilities being concatenated into a numerical representation of the document. These vectors are then used by a downstream classifier for detection, offering key advantages such as robustness to concept drift and inherent interpretability by capturing underlying malicious intent rather than surface-level features. As expected, within email-based approach, LLMs demonstrate great potential and can reach up to 97.46% accuracy.

URL-based approach focuses on analyzing the text of the uniform resource locator (URL) of a website directly targeting T1566.002 (Spearphishing Link). This method treats the URL as the main source of information, often processing it as sequential data and learning relationships among its characters or smaller sub-parts [138]. LLM-based methods can directly analyze the raw URL text, adopting a fine-tuning [114] or prompt engineering [120, 131, 136] method. LLMs can leverage their contextual understanding to identify suspicious patterns like minor character differences or fake brand names in domain names.

HTML-based phishing detection is another approach that examines a webpage's HTML source code to identify features indicative of phishing activity. This method examines elements such as HTML tags, their structure, layout consistency, textual content, and other features within the code that attackers might use to embed malicious functionalities. These models often require preprocessing to simplify lengthy code and fit within model token limits. In this regard, PhishLang [134] presents a lightweight and efficient antiphishing framework designed for direct operation on the user's device. PhishLang's methodology involves training on both website source code and URLs to detect phishing attempts.

Brand-based phishing detection approach leverages the immutable nature of genuine brand domains to achieve high precision and provide interpretable detection outcomes. It mainly involves two steps: First, identify the brand that the webpage under investigation is trying to impersonate, and second, compare the official URL of that brand to the URL of the webpage. This approach often requires the analysis of visual elements such as logos on screenshots and textual content within the HTML. As a result, multimodal language models are particularly well-suited for implementing this approach. Models adopting this approach

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

seek to overcome the limitations of traditional methods, which often depend on static, manually curated knowledge bases, and constrained to a single input modality, such as images or text [124, 127, 129, 130].

Because of their strong capabilities in natural language understanding, which are essential for processing emails, URLs, and HTML, as well as their rapidly improving multimodal reasoning that enhances brand-based and visually oriented phishing detection models, current LLM-based approaches can be positioned within the performance validation stage of the TRL framework. At this stage, the technology has demonstrated measurable effectiveness in controlled and semirealistic evaluation settings, yet still requires further refinement and large-scale operational testing before progressing toward full deployment readiness. Nevertheless, several challenges persist among the reviewed models, including token length limitations that require preprocessing of large raw inputs such as HTML content, as well as the high computational load and API costs associated with multimodal LLMs, particularly when processing visual data. Researchers are actively attempting to address these issues with ideas such as agentic approaches [139], optimized prompt engineering [136], and the development of efficient open-source models [118], with expectations that these limitations will be significantly mitigated in the near future.

Studies on phishing detection commonly construct their datasets by supplementing phishing samples with legitimate websites obtained from publicly available sources, thereby enabling a more representative evaluation of detection systems. For phishing data, widely used resources include PhishTank [116], a community-driven repository containing over 1.9 million verified phishing URLs, as well as email-based datasets such as the Nazario phishing corpus [112] comprising 5000 phishing emails, the IWSPA-AP training data [113] consisting of 5088 legitimate and 612 phishing emails, and Kaggle's fraudulent email corpus [123], which provides 4098 phishing emails. Conversely, legitimate and benign data are often sourced from extensive collections like Alexa [140] and Tranco lists [125] for URLs, or email archives such as Enron [141, 142].

5.4 | Vulnerability Detection

Vulnerability detection involves identifying weaknesses in software, systems, and configurations to prevent exploitation. It maps primarily to the NIST CSF functions Identify (ID), especially Asset Management (ID.AM) and Risk Assessment (ID.RA), and Detect (DE) for real-time scanning. In terms of MITRE ATT&CK, it helps mitigate Tactics such as TA0001 (Initial Access), TA0002 (Execution), and TA0004 (Privilege Escalation) by reducing exploitable attack surfaces. Historically, vulnerability detection relied on traditional methods such as manual code inspections, rule-based static analysis, and dynamic analysis. However, these approaches often suffer from significant limitations, including low accuracy, high false-positive rates, scalability issues with complex modern software, and a demanding need for extensive human expertise and time [143, 144].

To address these limitations, LLMs, with their sophisticated abilities in code comprehension, generation, and summarization, as well as their capacity to capture complex sequential patterns, provide promising solutions that surpass traditional methods [145]. Nonetheless, several studies have identified limitations associated with the direct use of general-purpose LLMs in this domain. These models frequently process source code as plain text, lacking domain-specific knowledge essential for accurate vulnerability identification. Their performance remains limited, frequently approaching the performance of a random baseline or a naive classifier [146–148]. As a result, researchers in this field have adopted several methods to enhance their LLM-based models, which we will examine in the remainder of this section.

A common approach in automated software vulnerability detection involves processing source code as raw, sequential text inputs for LLMs. This methodology typically entails fine-tuning pretrained LLMs on specialized, task-specific datasets to develop their proficiency in identifying vulnerable code patterns [149–153]. Moreover, domain-specific pretrained models, such as CodeBERT [154], CodeGen [155], and CodeT5 [156], are specifically developed by training on extensive corpora of programming languages, often combined with natural language, allowing them to capture both semantic and syntactic relationships within code. By leveraging these specialized models, researchers can develop frameworks that effectively capture complex code patterns, provide more fine-grained vulnerability localization, and handle large codebases, significantly enhancing the accuracy, integrity, and trustworthiness of software systems.

Another common approach in this domain involves extending beyond raw source code text by explicitly incorporating and leveraging the semantic, syntactic, and structural relationships inherent in the code. This approach is designed to overcome the limitations of existing methods that struggle with the complexity and diversity of modern codebases. In this regard, one strategy could be employing a knowledge distillation framework where specialized teacher models, trained on semantic and syntactic code relationships, guide a student LLM to internalize these structural nuances [157]. Another adopted strategy is to create an ensemble by fine-tuning multiple, diverse pretrained LLMs, each with unique strengths in understanding code semantics or structure, and then integrating their predictions using a meta-classifier [158].

Employing prompt engineering for vulnerability detection offers a cost-effective alternative to fine-tuning by harnessing the generalization capabilities of LLMs, enhancing interpretability, and mitigating the risk of catastrophic forgetting of pretrained knowledge [159–162]. In this context, Grace [159] integrates graph structural information and in-context learning to enhance LLMs' capability in software vulnerability detection. Similarly, SmartGuard [161] methodology involves leveraging LLMs to detect smart contract vulnerabilities by first retrieving semantically similar code snippets and then generating corresponding CoT for these snippets to facilitate in-context learning. Smart contracts are small programs deployed on the blockchain that automatically enforce terms and conditions between two untrusted parties. These contracts manage funds or data on the blockchain using predefined business logic. Detecting vulnerabilities in smart contracts is essential, as flaws in their code logic can severely compromise blockchain security. Unlike conventional software, smart contracts are immutable once deployed, making it difficult to fix vulnerabilities after deployment.

In a recent study [162], several researchers propose a novel approach to detect vulnerabilities by reframing it as reconstruction-based anomaly detection problem. The

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 5 | Overview of phishing detection models.

Reference	Name	Approach	Methods	LLMs	Datasets	*F1*
[111]	PHISHLM (2022)	Email-based	Fine-tuning, priming	GPT-2	[112, 113]	—
[114]	PhishBERT (2023)	URL-based	Pretraining from scratch, fine-tuning	BERT	[115, 116]	0.920
[117]	— (2023)	Email-based	Prompt engineering	Various models	Self-built	—
[118]	CyberGPT (2024)	Email-based	Fine-tuning	GPT-3.5, GPT-4	[119]	0.967
[120]	— (2024)	URL-based	Prompt engineering, fine-tuning	Various models	[121]	0.973
[122]	— (2024)	Email-based	Prompt engineering	Various models	[123]	
[124]	— (2024)	Brand-based	Prompt engineering	GPT-4, GeminiPro 1.0, Claude3	[125, 126]	—
[127]	KnowPhish detector (2024)	Brand-based	Prompt engineering, fine-tuning	GPT-3.5, XLM-RoBERTa	[125, 126, 128]	0.920
[129]	GEPAgent (2024)	Brand-based	LLM-based agents	GPT4, GPT-4V	[125, 126]	0.946
[130]	ChatPhish-detector (2024)	Brand-based	Prompt engineering	GPT-4V	[116, 125, 126]	0.993
[131]	— (2024)	URL-based, screenshot-based	Prompt engineering	Gemini 1.5 and GPT-4o	[132]	0.937
[133]	— (2024)	Email-based	Prompt engineering	GPT-3.5, GPT- 4, Gemini Pro	Self-built	0.910
[134]	PhishLang (2025)	URL-based, HTML-based	Fine-tuning	MobileBERT	[126, 135]	0.940
[136]	APOLLO (2025)	Email-based, URL-based	Prompt engineering	GPT-4o	[112, 123, 137]	0.999

underlying assumption is that deviations from this learned normal distribution indicate potential anomalies. As ANVIL does not require training, its reconstruction-based detection strategy demonstrates strong generalization across different projects. Another study [163] conducts an empirical study and highlights the limitations of LLMs in distinguishing between vulnerable code and its semantically similar but benign patched counterparts. Subsequently, the study introduces a RAG-based framework designed to address this limitation.

The burgeoning volume of research on software vulnerability detection is primarily motivated by the escalating prevalence and inherent complexity of software vulnerabilities within contemporary, rapidly evolving systems [144]. While general-purpose LLMs often exhibit limitations in direct vulnerability detection [146, 148], their effectiveness is considerably enhanced through the strategic integration of domain-specific knowledge by employing advanced contextual understanding techniques [158, 159], explainability mechanisms [163, 164], as well as specialized fine-tuning [150, 153] and multiagent systems [165, 166]. Moreover, this surge in LLM-based software vulnerability detection research is driven by their impressive capabilities in code comprehension and generation, which effectively address the limitations of traditional detection methods [143].

The proliferation of code-based LLMs, the substantial volume of research publications in this domain, and the growing maturity of standardized benchmarks for security evaluation have collectively positioned LLM-based vulnerability detection within the performance validation stage of the TRL framework. Nevertheless, this domain is not without its challenges. As shown in Table 6, most datasets are structured at the function level, hindering the precise detection of vulnerable codes down to the individual line. Furthermore, a common simplification is that many studies assume that there is only one vulnerability in a code snippet, overlooking real-world complexities where multiple vulnerabilities may coexist within a single file [161]. Overcoming these limitations through the curation of high-quality, fine-grained, and diverse datasets, is essential for LLMs to transition from academic exploration to reliable, practical deployment in automated software vulnerability detection.

Benchmarks in this field originate from real-world projects, synthetic examples, or a combination of both. Most of these benchmarks are based on the C programming language. This predominance can be attributed to the widespread use of C/C++ in technological infrastructure, their susceptibility to critical memory and resource management vulnerabilities, and the greater availability of public benchmark datasets for these languages. Notable examples include VUDENC for Python code (1009 vulnerability-fixing commits) [181], as well as Big-Vul (3754 vulnerabilities) [167], CVEfixes (11,873 CVEs) [182], and SARD (over 170,000 test cases) [168], which are predominantly focused on C/C++ vulnerabilities.

5.5 | Others

There are several studies exploring the integration of LLMs into tasks beyond those discussed earlier. Nevertheless, their number remains limited, and further research is required in these areas. Table 7 presents the various LLM-based models employed in the detection phase. MalBERT [183] and SeMalBERT [186] both

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

leverage BERT for static analysis-based malware detection, aiming to overcome the limitations of traditional methods that struggle with imprecise representations and a lack of contextual understanding in identifying malicious software. Effective spam detection is also critical for cybersecurity and security teams as unsolicited messages frequently serve as entry points for cyberattacks. Within the MITRE ATT&CK framework, such attacks typically align with the Initial Access tactic (TA0001), which encompasses techniques used to gain an initial foothold within a network. Beyond this, the widespread delivery of spam emails and SMS disrupts user productivity, wastes network resources, and introduces a myriad of security risks to individuals and organizations [187, 189, 190]. Furthermore, another study [188] introduced a model to detect multistage attacks. These complex attacks involve multiple steps to achieve an attacker's ultimate goal, but their detection is hindered by the sheer volume of alerts from diverse sources, their heterogeneous formats, and often weak correlations. Several researchers have also investigated the application of LLMs in web content classification [185]. This task involves categorizing web resources, such as URLs, into predefined content types to manage access, enforce policies, and ensure network security and regulatory compliance within organizations. It ultimately fosters a secure and professional work environment, enhancing operational efficiency and protecting the organization's reputation.

Answer to RQ2: In the detection phase, among the LLM-based models proposed for various tasks, log anomaly detection and vulnerability detection have been extensively studied and achieved satisfactory performance. In the areas of NID and phishing detection, models with acceptable performance on specific datasets have been developed, but additional research and evaluation on larger datasets are necessary. Finally, malware detection and spam detection represent tasks with limited existing work, although their development is expected to grow in the near future.

6 | RQ3: How Are LLMs Adapted for the Analysis Phase of SOCs?

The Analysis phase is a critical component that supports diverse tasks spanning the NIST Identify, Detect, and Respond functions. It begins once an alert has been generated and involves a deep investigation to validate the threat, understand its context, and inform the subsequent response. This phase includes tasks such as alert triage, threat intelligence enrichment, and incident contextualization. LLMs are particularly valuable here, as they can process and synthesize vast amounts of structured and unstructured data to provide analysts with actionable insights.

6.1 | Domain-Specific Language Models

Domain-specific language models are specialized variants designed to effectively comprehend and process textual data within a particular domain, such as cybersecurity. They are superior to general-purpose models such as BERT, which, despite their broad applicability, often fail to effectively process domain-specific text due to substantial differences in terminology, writing styles, and word distributions [191, 192]. For example, the meaning of the word virus in general models might be a microscopic organisms that can infect hosts, like humans, while in fact, it is a type of malware in the cybersecurity [193]. Moreover,

TABLE 6 | Overview of vulnerability detection models.

Reference	Name	Method	LLM	Detection level	Datasets	*F*-1
[149]	Linevul (2022)	Fine-tuning	CodeBERT	Line level	[167]	0.920
[150]	VulDetect (2023)	Fine-tuning	GPT-2	File level	[168, 169]	0.924, 0.829
[151]	LLMAO (2024)	Fine-tuning	CodeGen	Line level	[170]	—
[159]	GRACE (2024)	Prompt engineering	CodeT5, GPT-4	Function level	[167, 170, 171]	0.651, 0.431, 0.355
[157]	SAFE (2024)	Knowledge distillation	RoBERTa, CodeT5	Function level	[170–172]	0.667, 0.490, 0.707
[164]	LProtector (2024)	RAG	GPT-4o	Function level	[167]	0.335
[173]	— (2024)	Fine-tuning	BERT	Function level	[174]	0.879
[158]	EnStack (2024)	Fine-tuning	CodeBERT, GraphCodeBERT, UniXcoder	Function level	[175]	0.823
[160]	DLAP (2025)	Prompt engineering	LineVul, GPT-3.5	Function level	Self-built	0.584
[152]	SequreQwen (2025)	Fine-tuning	CodeQwen1.5	Function level	[152]	0.950
[176]	LLM-VULEXP (2025)	Prompt engineering, fine-tuning	CodeLlama, GPT-3.5	Function level	[169, 174]	0.924
[153]	SecureFalcon (2025)	Fine-tuning	Falcon	File level, function level	[177]	0.940, 0.920
[161]	SmartGuard (2025)	Prompt engineering	CodeBERT, GPT-3.5	Function level	[178]	0.949
[162]	ANVIL (2025)	Prompt engineering	CodeLlama	Function level, line level	[179, 180]	(ROC) 61.8%
[163]	VulRAG (2025)	Prompt engineering, RAG	Various models	Function level	[163]	(Pair Acc) 0.320

Abbreviations: Acc, Accuracy; ROC, area under the receiver operating characteristic curve.

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 7 | Overview of LLM-based models within different detection tasks.

Reference	Name	Methods	LLM	Downstream tasks	*F*-1
[183]	MalBERT (2021)	Fine-tuning	BERT	Malware detection	0.955
[184]	— (2022)	Representation learning	BERT	Anomalous HTTP requests detection	0.981
[185]	— (2023)	Knowledge distillation	Various models	Web content classification	(Acc) 0.460
[186]	SeMalBERT (2024)	Representation learning	BERT	Malware detection	0.988
[187]	IPSDM (2024)	Fine-tuning	DistilBERT, RoBERTa	Spam detection	0.621
[188]	MAD-LLM (2024)	Prompt engineering	GPT-4	Multistage attack detection	0.875
[189]	— (2025)	Prompt engineering	GPT-3.5	Spam detection	0.790
[190]	Explainable detector (2025)	Fine-tuning	DistilBERT, RoBERTa	Spam detection	0.994

Abbreviation: Acc, Accuracy.

incorporating domain-specific LMs into the architecture of analytical models in this field often leads to improved performance compared to their general-purpose counterparts [194, 195].

Table 8 lists domain-specific LMs, which are widely adopted among researchers in the field of cybersecurity. These models can be categorized based on their training methods: continual pretraining [191, 193, 196] and pretraining from scratch [192]. A Continual pretraining method typically requires significantly less domain-specific data and time compared to pretraining from scratch. However, because the volume of domain-specific data is relatively small in comparison with the data used in general pretraining, the semantic representations of many common words remain largely unchanged from those in general-purpose models. In SecureBERT [196], they attempt to address this challenge by introducing a small noise to the initial pretrained weights of mutual tokens from a general language model, and it is capable of outperforming RoBERTa-Large, despite the latter having nearly three times as many parameters.

6.2 | CTI Extraction

CTI refers to evidence-based knowledge encompassing context, mechanisms, indicators, potential impacts, and actionable recommendations related to existing or emerging threats to assets [197]. SOC teams must continuously consume and analyze a deluge of CTI from sources such as security company content platforms [198, 199], security news [200, 201], and influential personal security blogs [202, 203] to understand the evolving threat landscape. This is a time-consuming task that LLMs are uniquely suited to automate. LLM-based systems can ingest and summarize vast quantities of unstructured CTI. This provides a proactive edge in the NIST "Respond" function by enabling teams to anticipate threats. For instance, an LLM can process dozens of reports about a new malware campaign and synthesize the key TTPs, identifying that the campaign primarily uses T1566.001 (Spearphishing Attachment) for initial access, followed by T1055 (Process Injection) for execution. This automated synthesis allows analysts to quickly grasp the full scope of a threat actor's methodology and update detection rules accordingly, a task that would otherwise take hours of manual work.

As shown in Table 9, TTP classification and threat entity/relation extraction are the two main objectives of the proposed models in this field. Models with the aim of only extracting relevant TTPs usually involve LLMs to understand the context of sentences and then map them to relevant TTPs using deep learning models [194, 204, 211]. For instance, TTPHunter [194] utilizes the combination of RoBERTa and linear classifier to predict the corresponding TTP class for each sentence. It also employs the thresholding technique to the classifier's confidence score to filter out irrelevant sentences that do not strongly describe any TTPs. The proposed model was validated using two distinct datasets: a sentence-based dataset comprising 8387 samples derived from the MITRE ATT&CK knowledge base, and a document-based dataset consisting of 50 real-world threat reports. Experimental results yielded peak performance, with F1 scores of 88% on the sentence dataset and 75% on the document dataset. Since TTPHunter is limited to only identifying 50 most frequent TTPs, the authors of it propose a refined version, called

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 8 | Overview of cybersecurity language models.

Reference	Name	Method	LLM	Downstream tasks	Sources
[191]	CyBERT (2021)	Continual pretraining	BERT	Named entity recognition, threat classification, cybersecurity knowledge graph	Krebs on security, CVEs, APTNotes
[196]	SecureBERT (2022)	Continual pretraining	RoBERTa	Sentiment analysis, named entity recognition	Websites, security reports, whitepapers, books, articles, surveys, videos (YouTube)
[192]	CTI-BERT (2023)	Pretraining from scratch	BERT	ATT&CK technique classification, IoT app description classification, malware sentence detection, malware attribute classification, coarse-grained and fine-grained security entities, token-type classification	Attack description (MITRE ATT&CK), security textbook, academic paper, security wiki, threat report, vulnerability description (CVEs and CWEs)
[193]	CySecBERT (2024)	Continual pretraining	BERT	Clustering, word similarity, named entity recognition, relevance classification, specialized CTI classification	Blogs, arXiv, NVD, twitter

TTPXHunter [211] to address this limitation and improve its performance.

Several models have been designed to extract threat entities and their relationships in conjunction with TTPs. A common approach involves fine-tuning models to first identify threat entities and subsequently determine the relationships between them, typically within a single sentence [206, 207, 210]. Ignoring relationships outside of a sentence and the high amount of processing required to examine each pair of threat entities in a sentence are some existing challenges among the models with fine-tuning approach. These limitations and applicability across diverse datasets have led recent studies to show an increased tendency toward using prompt engineering [208, 209, 212, 213]. In this regard, the aCTIon framework [208] employs a two-step LLM querying methodology, beginning with an iterative summarization phase to condense input and filter noise, followed by an extraction and self-verification phase to identify and classify entities and attack patterns. This approach utilizes distinct pipelines for general entity and relation extraction versus specific attack pattern identification, leveraging various prompt strategies to achieve these tasks effectively. For specific entity types, the model achieved an average $F1$ score of 72% for malware and 80% for threat actors. Furthermore, the system demonstrated strong results in relationship extraction, achieving an average 88% precision and 73% recall for "targets" and "uses" relations. Moreover, another study [212] proposes a framework for automated extraction of detection rule candidates from cloud-based open-source CTI. Their study distinguishes itself from prior models through several advancements like by processing both textual and visual data, specifically targeting cloud security environments, and generating its output as Sigma rules, which ensures that the extracted intelligence is both relevant and immediately actionable for integration into SIEM systems.

This field currently faces several challenges that must be addressed prior to model development. One such challenge is the dynamic nature of CTI frameworks, such as MITRE ATT&CK, which undergo continuous updates. This constant evolution renders traditional fine-tuning approaches less suitable over time. Compounding this issue is a critical limitation observed in the existing LLM-based systems: their reliance on a one-to-one classifier model. Such a design restricts the mapping of a given sentence to only a single TTP, thereby struggling to capture the inherent ambiguity and complexity of real-world cyberattack descriptions, where a single event may correlate with multiple TTPs or distinct campaign stages. Consequently, these models may overlook critical insights or misclassify information, preventing a comprehensive understanding of multifaceted threat behaviors. As a result of these obstacles, LLM-based methods in this domain remain at the emerging stage of technological readiness. Addressing these challenges is therefore crucial for advancing truly intelligent and adaptive CTI automation.

Another task related to this domain that can be elevated by LLMs is CTI generation. This task involves producing comprehensive cybersecurity reports in natural language by utilizing structured representations of threat intelligence as input. This process offers the advantages of being easily comprehensible to personnel without specialized expertise and facilitating straightforward dissemination across the organization. In this context, AGIR [214] generates reports by first parsing a STIX graph into

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 9 | Overview of LLM-based CTI extraction models.

Reference	Name	Method	LLM	Extracted CTI types	Output
[204]	TIM (2022)	Representation learning, ML-based classification	Sentence-BERT	TTPs	STIX, sigma rules
[205]	TriCTI (2022)	Fine-tuning	CBERT, BERT	IOCs, campaign stages	Labeled IOCs
[206]	Vulcan (2022)	Fine-tuning	BERT	Threat entities, threat relationships, TTPs	Graph database
[194]	TTPHunter (2023)	Fine-tuning	BERT, RoBERTa	TTPs	Knowledge graph
[207]	LADDER (2023)	Fine-tuning	BERT, RoBERTa, XLM-RoBERTa	Threat entities, threat relationships, TTPs	Knowledge graph
[208]	aCTIon (2023)	Prompt engineering	GPT-3.5	Threat entities, threat relationships, TTPs	Knowledge graph
[209]	AttackKG+ (2024)	Prompt engineering	Various models	Threat entities, threat relationships, TTPs	Knowledge graph
[210]	LLM-TIKG (2024)	Prompt engineering, instruction tuning	GPT-3.5, Llama 2	Topic, threat entities, threat relationships, TTPs	Knowledge graph
[211]	TTPXHunter (2024)	Fine-tuning	SecureBERT	TTPs	STIX
[212]	LLM CloudHunter (2025)	Prompt engineering	GPT-4o	Threat entities, threat relationships, TTPs	Sigma rules
[213]	CTINEXUS (2025)	Prompt engineering	GPT-4	Threat entities, threat relationships, TTPs	Knowledge graph

17 42, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

a template-based structure and subsequently refining the resulting text for fluency and quality using ChatGPT. Automating CTI generation can bring benefits such as reduction of manual effort, time, and expert demands [215].

A growing body of CTI research relies on datasets derived from a range of unstructured sources, including security blogs, incident reports, social media posts, and threat intelligence feeds. Among the most frequently cited public resources for CTI extraction are Krebs on Security [202], Threat Post [216], and The Hacker News [217], each of which provides timely, detailed accounts of real-world cyber incidents and adversary behaviors.

6.3 | Knowledge Base Alignment

Knowledge base alignment involves linking diverse cybersecurity knowledge bases such as CVEs, CWEs, CAPEC, and the MITRE ATT&CK framework [218]. This process is essential given the increasing complexity of modern computing frameworks and the rising number of reported cybersecurity vulnerabilities. Although each of the mentioned knowledge sources offers valuable information for SOC teams, they exhibit inherent limitations when considered independently [219]. For example, CVEs often provide little or no information on how to combat the vulnerability, necessitating linkage to frameworks like ATT&CK [220]. Moreover, mapping between this sources could provide a holistic view of the entire cyber landscape, encompassing threats, vulnerabilities, weaknesses, attack patterns, and affected systems and help cybersecurity defense teams to better prioritize alerts. Nevertheless, performing this task manually is inefficient since it requires extensive effort, considerable time, and substantial domain expertise. To address these challenges, several studies have explored automating this process utilizing LLMs.

As demonstrated in Table 10, most mapping frameworks focus on establishing connections between CVEs and MITRE ATT&CK techniques. Combining these two complementary resources is vital for enhancing strategic analysis, contextualizing security alerts, supporting product security evaluations, and enabling proactive threat hunting and risk prioritization. Moreover, the frequent reference to the MITRE ATT&CK framework can also be explained by its broad integration into industry-standard SIEM tools. In this regard, some studies [7, 195] utilize semantic role labeling techniques to extract attack vectors and as a result, reducing noise by focusing on them and circumventing the reliance on preannotated datasets. Next, they fine-tune a LLM to generate semantically meaningful vector representations of these extracted attack vectors. Crimson [224] is another framework, the main idea of which revolves around retrieval-aware training (RAT). RAT integrates real-time data retrieval directly into the LLM training process, enriching both the training and inference phases with current and relevant cybersecurity data. This mechanism ensures that the model is up-to-date with the latest threats and information. Their evaluation results show that Crimson achieves accuracy comparable to GPT-4 while producing fewer hallucinations, which is noteworthy given that it contains nearly 250 times fewer parameters than GPT-4. Furthermore, several researchers [222] propose a two tier stage model for mapping between vulnerabilities and attack patterns. In first stage, it maps CVEs to CWEs through the V2W-BERT [221] model, which consists of RoBERTa in a Siamese network architecture. In the second stage, T5 generates the

TABLE 10 | Overview of LLM-based mapping models.

Reference	Name	Method	LLM	From	To	*F1*
[220]	CVET (2021)	Fine-tuning	RoBERTa	CVE	MITRE ATT&CK tactics	0.762
[221]	V2W-BERT (2021)	Continual pretraining, fine-tuning	BERT	CVE	CWE	0.972
[222]	VWC-MAP (2022)	Fine-tuning	RoBERTa, T5	CVE	CAPEC	—
[223]	CVE2 ATT&ACK (2022)	Fine-tuning	SecBERT	CVE	MITRE ATT&CK techniques	0.478
[7]	SMET (2023)	Fine-tuning	Sentence-BERT	CVE	MITRE ATT&CK techniques	—
[195]	TTPpredictor (2023)	Fine-tuning	SecureBERT	CVE	MITRE ATT&CK techniques	0.980
[224]	Crimson (2024)	Retrieval-aware training	Claude2, GPT-4, LLaMA-2	CVE	MITRE ATT&CK techniques	0.729
[225]	— (2024)	RAG	Llama 3	CAPEC	MITRE ATT&CK techniques	—
[226]	— (2024)	RAG	GPT-3.5	TTPs	MITRE ATT&CK tactics	0.950
[227]	— (2025)	Prompt engineering	GPT-4, Claude-3, Gemini	NIDS rules	MITRE ATT&CK tactics and techniques	—

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

CAPEC text description using the CWE text description as input, and ultimately, only a similarity measure with attack patterns is required. Experimental results, which were cross-validated by cybersecurity experts, demonstrate the framework's effectiveness, achieving up to 87% accuracy for mapping vulnerabilities to weaknesses and 80% accuracy for mapping weaknesses to attack patterns. Requirement for intensive computational resources such as multi-GPU parallel processing, the limited size of the evaluation dataset, and reliance on manual qualitative assessment are some potential limitations of this model.

Besides CVEs, there are other knowledge sources from which establishing mappings could be beneficial. For instance, some researchers conducted a comparative analysis to address the significant challenge of cybersecurity practitioners face in understanding the complex and often ambiguous descriptions within the ATT&CK framework [226]. Moreover, linking NIDS rules with MITRE ATT&CK techniques is another important mapping task mentioned in a recent study [227]. Automation of this task aims to improve the efficiency of SOC alert analysis while reducing the operational burden on analysts.

LLM-based methods for mapping cybersecurity knowledge bases remain in an emerging stage within the TRL framework due to their limited and inconsistent performance across complex, real-world scenarios. In particular, these methods often do not perform well enough in tasks requiring precise entity disambiguation, multihop reasoning, and robust handling of noisy or incomplete threat intelligence data. Furthermore, challenges related to evaluation standardization, reproducibility, and integration with operational cybersecurity systems continue to hinder their progression toward higher readiness levels.

6.4 | Question-Answering (QA) Models

SOC analysts face a critical operational challenge, tasked with processing vast and complex volumes of CTI under extreme time pressure, where errors can lead to severe organizational damage. This burden forces highly skilled experts to devote significant time to labor-intensive data sifting, thereby motivating the adoption of QA models to augment analyst capabilities and streamline the threat intelligence workflow. QA models are computational systems that allow users to obtain answers to their questions expressed in natural language, serving as an advanced approach to information retrieval. For small- and medium-sized enterprises, QA models are crucial for providing intuitive and user-friendly access to complex cybersecurity information, compensating the limited technical expertise and personnel, and enabling rapid, informed decision-making around the clock [228].

Traditional QA models faced significant limitations primarily due to their struggle to fully comprehend the user's query context and content meaning. These systems often relied on predefined rules and patterns, which made them inflexible and unable to adapt to diverse user interactions or handle complicated and unclear questions effectively. With the advent of LLMs, the first idea that came to the minds of many researchers was to use these general-purpose models as chatbots in the field of cybersecurity. However, this idea also failed to meet expectations due to persistent problems such as tendency to hallucinate, outdated knowledge, and high sensitivity to engineering prompts [229, 230].

As can be seen in Table 11, RAG-based methods are the most popular approach among researchers in this field. This approach directly mitigates hallucination and provides access to current information, crucial for a rapidly evolving field like cybersecurity where factual accuracy and timeliness are paramount. Furthermore, RAG models can provide the source of their answers, enhancing transparency and allowing cybersecurity analysts to verify the information, which is vital for informed decision-making [232, 239]. A prevalent architectural pattern among RAG models involves the simultaneous or sequential use of multiple knowledge bases to retrieve pertinent information for distinct use cases [234, 237]. In this context, LOCALINTEL [237] integrates publicly available global CTI with private organizational knowledge to generate context-aware analytical responses. Upon receiving a prompt from a SOC analyst, it retrieves relevant information from global repositories (e.g., CVE and NVD) and local databases, enabling the LLM to produce a comprehensive and organization-specific response. Depending on the implementation, a two-database approach offers several advantages, including enhanced efficiency and faster retrieval, improved accuracy through organization-specific contextualized responses, and better support for data segregation and privacy. Moreover, the quantitative evaluation of the study, conducted using the RAGAS framework, demonstrated a mean answer relevancy score of 0.95, indicating its capability to consistently provide accurate and relevant intelligence for SOC analysts.

Fine-tuning on large-scale corpora of question–answer pairs represents another approach within QA models. In a work [233], they collect over ten thousand bilingual (English and Chinese) cybersecurity incident websites from leading vendors and news outlets. They also employ Select-Instruct to create the instruction corpus, wherein a raw corpus is combined with prompts to guide GPT-4 in selecting a task from the pool and subsequently generating structured query–response pairs for the selected task, accompanied by a reasoning process. The final dataset contains nearly 90,000 samples encompassing 28 well-designed tasks, categorized into 13 understanding tasks (e.g., key entity recognition and malware feature extraction) and 15 generation tasks (e.g., summary generation and security alert generation). SecKnowledge [238] is another cybersecurity instruction dataset that involves a multiphase generation process. The first-generation step focuses on building high-quality instructions based on predefined schemas established through extensive domain expertise and in-depth analysis of diverse security datasets and then on, the second step, expands these instructions through the hybrid synthetic content–based data generation process. The authors also introduce SecKnowledge-Eval, a comprehensive benchmark consisting of 15 diverse datasets, including eight newly developed tasks and seven public benchmarks. It is designed to assess both knowledge and analytical understanding, featuring unique components such as Adversarial MITRE ATT&CK as well as specialized tasks for relationship prediction and technical impact classification.

Among studies, some employ traditional text evaluation metrics such as ROUGE, BLEU, or semantic similarity, while others utilize LLMs as evaluators. However, these approaches only assess the final generated answer and are insufficient for comprehensively evaluating RAG-based models. To address this limitation, recent work has adopted RAGAS [240], a framework specifically designed for assessing RAG systems through metrics

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 11 | Overview of question-answering models.

Reference	Name	Method	LLM	Generation method	Key features
[231]	Cyber Sentinel (2023)	Prompt engineering	GPT-4	Human-curated	Designed to handle a number of security actions
[232]	— (2024)	RAG	Mistral	Human-curated	Focused on cyberattack investigation and attribution
[233]	SEVENLLM (2024)	Fine-tuning	GPT-4, Llama 2, Qwen-1.5	LLM-generated	Creation of bilingual instruction corpus
[234]	CyberGuardian (2024)	Fine-tuning, RAG	Llama 2	Human-curated	Has a plugin architecture
[235]	— (2024)	Fine-tuning, RAG	Llama 3	Hybrid	Creation of a Q&A dataset based off the MITRE ATT&CK
[236]	IntelBot (2024)	Prompt engineering, RAG	BERT, GPT-3.5	Human-curated	Ensemble retriever
[237]	LOCALINTEL (2025)	LLM-based agents, RAG	GPT-3.5	Human-curated	Leveraging global and local knowledge
[238]	CYBERPAL.AI (2025)	Fine-tuning	Mistral, Llama-3, Phi-3	Hybrid	Comprehensive instruction dataset
[239]	ChatNVD (2025)	Prompt engineering, RAG	Various models	Human-curated	Streamline vulnerability analysis for diverse users

Note: Generation method column describes the method of constructing questions that form the dataset for evaluating or training models.

such as answer relevance, context precision, and faithfulness. Furthermore, as LLMs are increasingly applied to complex, domain-specific tasks like cybersecurity, the development of robust and diverse evaluation benchmarks has become crucial to accurately assess their capabilities and performance. CyberMetric [241], SecKnowledge-Eval [238], and SEVENLLM-Bench [233] are some proposed cybersecurity benchmarks in this domain. These benchmarks play a vital role in advancing the field by providing standardized tools to measure LLMs' understanding, reasoning, and practical applicability in real-world cybersecurity scenarios, especially regarding CTI.

LLM-based methods for cybersecurity QA models are positioned within the operational adaptation stage of the TRL framework due to their demonstrated capability in handling diverse and context-rich queries. This progression is largely supported by the inherent suitability of QA tasks for LLMs, as such models are designed to process natural language inputs and generate coherent, context-aware responses. Additionally, their adaptability to domain-specific knowledge through fine-tuning and retrieval augmentation has enabled practical deployment in real-world cybersecurity applications, despite remaining challenges in reliability and evaluation consistency.

6.5 | Log Analysis

Log analysis is the process of examining logs to identify events of interest. It primarily maps to the Respond function of the NIST CSF when employed to investigate and mitigate incidents. Given the sheer volume of these logs, manual inspection is impractical, even with basic search tools, leading to a fast-growing demand for automatic log analysis frameworks [242].

As shown in Table 12, the root cause analysis (RCA) task is the most popular topic among researchers in this field. This task aims to identify the underlying cause of abnormal behavior. RCA is a critical but often laborious, error-prone, and challenging task for on-call engineers (OCEs), especially given the escalating scale and complexity of modern cloud systems and the huge volume of logs they generate [244]. A shared characteristic among all models proposed for this task is their focus on mitigating the cumbersome effects of these challenges. Fine-tuning has not been widely adopted for this task due to a set of demanding requirements for effective RCA. These requirements include the ability to interact with external environments for missing data, synthesize high-volume, multisource information into actionable insights, adapt to evolving threats without costly retraining, and provide detailed explanations rather than just a final prediction. LLM-based agents inherently satisfy these prerequisites, making them a more suitable architectural choice for such complex models [245, 246, 249, 254].

RCACopilot [244] is an on-call system powered by LLMs for automating the RCA of cloud incidents. RCACopilot is designed to match incoming incidents to corresponding incident handlers based on their alert types, aggregate critical runtime diagnostic information from multiple sources, predict the incident's root cause category, and then provide an explanatory narrative. Although this model has been deployed at Microsoft, it exhibits certain limitations, including its dependence on manually crafted predefined handlers and its focus on predicting general root cause categories rather than precise diagnoses. In a study [245], they propose a model to overcome these limitations. They also

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

address the inability to dynamically collect additional diagnostic information by utilizing LLM-based agents, which can reason, plan, and interact with the external environments. Furthermore, the study presents a practical case study with an Azure Fundamental Team at Microsoft. RCAgent [249] is another framework developed to operate on internally deployed models, with the objective of enabling privacy-preserving applications in industrial settings. This framework adopts a multiagent architecture and addresses the limited context length of LLMs through observation snapshot key (OBSK). OBSK, instead of feeding the entire, lengthy observation directly to the main agent, only presents a short summary of the observation. Along with this summary, it provides a unique hash ID, called a snapshot key that acts like a reference number for the full, detailed observation. The study also conducted extensive experiments on Alibaba Cloud's Real-time Compute Platform for Apache Flink, demonstrating that RCAgent outperforms the standard ReAct paradigm in root cause prediction, solution generation, and responsibility assignment.

Failure diagnosis constitutes another crucial task within the analysis stage, involving the classification of specific types of anomalies. In this regard, there are several models designed for the efficient and accurate extraction of critical fault-indicating details from system logs [250, 253]. Moreover, CrashEventLLM [247] employs fine-tuning to predict system crashes and their potential causes. Failure prediction adopts a proactive approach by aiming to generate early warnings concerning potential system failures. This framework was evaluated using Llama2-7B and Llama2-13B models fine-tuned on a small annotated dataset of 100 observation-label pairs and validated on a separate set of 40 pairs. Quantitative analysis using ROUGE metrics showed that both models achieved an $F1$ score of approximately 0.39 for crash time prediction, whereas for crash cause prediction, the Llama2-7B model surprisingly outperformed the larger Llama2-13B model with an $F1$ score of 0.46 compared to 0.34. Another study [252] highlights the dual challenges in automated log analysis: the limited capabilities of SLMs like BERT on complex logs, and the computational inefficiency and high cost of LLMs such as ChatGPT. To balance performance and inference costs, they propose a framework that integrates both models by strategically assigning LLMs to handle complex log instances, while simpler logs are processed by fine-tuned SLMs. Evaluations were conducted across six log analysis tasks, including anomaly detection, failure identification, and root cause ranking. The results demonstrate that AdaptiveLog achieves robust performance while reducing LLM-related costs by an average of 73%.

LLM-based methods within the log analysis stage occupy an intermediate position in the TRL framework, reflecting ongoing maturation alongside partial real-world integration. Some of these methods have been adopted in operational environments for noncritical tasks and are primarily utilized as assistants rather than autonomous decision-makers. Limitations in reliability, interpretability, and robustness when handling large-scale and heterogeneous log data continue to constrain their advancement toward fully autonomous deployment.

6.6 | Risk Assessment

Risk assessment involves evaluating an organization's assets, vulnerabilities, and threat landscape to determine potential cybersecurity risks and prioritize mitigation efforts. It primarily maps to the NIST CSF functions Identify (ID), particularly Asset Management (ID.AM) and Risk Assessment (ID.RA), and can also support Detect (DE) by guiding monitoring priorities. A recent study [255] investigates the use of LLMs with CoT prompting to automate cybersecurity risk assessments for embedded systems and compares it with expert's risk assessments. The findings demonstrate the potential of LLMs with CoT prompting to not only automate but also enhance the consistency and reliability of cybersecurity risk assessments in operational technology environments, offering a foundation for AI-assisted risk evaluations. Moreover, ExBERT [256] and CVEDrill [257] are two proposed framework to automate cybersecurity vulnerability analysis. ExBERT is an enhanced BERT model specifically designed for predicting whether a vulnerability will be exploited or not. CVEDrill also automates the prediction of CVSS vectors and classifies CVEs into the appropriate CWE hierarchy directly from unstructured textual descriptions of vulnerabilities. This enables organizations to implement cybersecurity countermeasure mitigation with enhanced accuracy and timeliness, surpassing the capabilities of general-purpose tools like ChatGPT in this specialized domain.

Answer of RQ3: In the analysis phase, nearly all tasks still require further research before they can be fully relied upon with confidence. However, QA and log analysis models may serve as a supplementary knowledge source for minor or noncritical tasks in real-world operations.

7 | RQ4: How Are LLMs Adapted for the Response Phase of SOCs?

Despite the deployment of advanced detection tools, their inherent operational limitations and incomplete organizational coverage due to resource constraints render comprehensive detection infeasible. Consequently, some incidents remain unavoidable, underscoring the necessity for organizations to establish a structured incident response (IR) plan. The NIST CSF formalizes the Respond phase as the core function of developing and implementing appropriate activities to take action regarding a detected cybersecurity incident, specifically designed to contain the impact of potential incidents through coordinated response actions.

7.1 | Vulnerability Repair

Automatic vulnerability repair (AVR) is an area of growing importance in software engineering, focused on automatically fixing security flaws in source code. It primarily maps to the NIST CSF functions Protect, through secure configuration and patch management, and Respond when remediating active vulnerabilities. The GitHub 2020 security report found that it takes approximately 4.4 weeks to release a fix after a vulnerability is identified [258]. Although AI-driven automation tools like GitHub Copilot have demonstrated substantial improvements in developer productivity for functional code generation, their effectiveness in vulnerability repair remains limited. In some cases, these tools may even introduce security flaws. This limitation is largely attributed to their pretraining on publicly available code repositories, which frequently contain vulnerable code patterns [259]. Therefore, by accelerating the fixing process, AVR minimizes the exposure of software systems to potential attacks,

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 12 | Overview of LLM-based log analyzing models.

Reference	Name	Method	LLM	Downstream tasks
[243]	— (2023)	Prompt engineering, fine-tuning	GPT-3.5	Failure mode classification
[244]	RCACopilot (2024)	Prompt engineering	GPT-4	Root cause analysis
[47]	— (2024)	LLM-based agents	Various models	Alert labeling
[245]	— (2024)	LLM-based agents	GPT-4	Root cause analysis
[246]	— (2024)	Prompt engineering	GPT-4	Root cause analysis
[247]	CrashEventLLM (2024)	Fine-tuning	Llama-2	Failure prediction
[248]	LogConfigLocalizer (2024)	Prompt engineering	GPT-4	Localize configuration errors
[249]	RCAgent (2024)	LLM-based agents	Vicuna	Root cause analysis
[250]	LoFI (2024)	Prompt tuning	UniXcoder	Failure diagnosis
[251]	COMET (2024)	Prompt engineering	GPT-4	Incident triage
[64]	LogLM (2025)	Instruction tuning	GPT-4, LLaMA-2	Log interpretation, root cause analysis
[252]	AdaptiveLog (2025)	Fine-tuning, prompt engineering	BERT, GPT-3.5	Failure identification, module classification, level prediction, log and description semantic matching, log and possible cause ranking
[65]	LUK (2025)	Multiexpert collaboration, continual pretraining, fine-tuning	Various models	Failure identification, module classification, fault phenomenon identification, log and description semantic matching, log and possible cause ranking
[253]	ScalaLog (2025)	RAG	GPT-3.5	Failure diagnosis
[254]	Flow-of-Action (2025)	LLM-based agents	GPT-3.5, GPT-4	Root cause analysis

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

enhances overall security, and offers a promising solution to address vulnerabilities, thereby significantly reducing the burden on security experts.

Fine-tuning is the most common and often preferred method for AVR models. Despite the massive scale and pretraining of LLMs, they often struggle with the specialized domain of vulnerability repair without further adaptation, as their generic knowledge is not sufficient for this complex task [260]. Additionally, vulnerability repair datasets are typically small and scarce, making traditional deep learning approaches difficult to train effectively from scratch [258]. Fine-tuning, especially when coupled with transfer learning from larger, related bug-fixing corpora, allows models to acquire generic knowledge from extensive codebases and then specialize it for the unique patterns and intricacies of security vulnerabilities, overcoming the data scarcity problem and yielding more reliable performance [261]. This process enables models to learn task-specific knowledge and adapt to the precise output formats required for generating accurate and compliable patches. Conversely, prompt engineering alone is generally ineffective for models like GPT-4 in the context of vulnerability repair [148]. Studies demonstrate these general-purpose models achieve very low accuracy, significantly trailing fine-tuned models [148, 261, 262]. Therefore, while prompting can offer some modest improvements by providing security context, it does not compensate for the lack of specialized training needed for robust vulnerability repair [263].

VulRepair [264] represents one of the earliest deployments of LLMs in this domain. A notable limitation of VulRepair, like other transformer-based models, is its struggle to handle lengthy vulnerable code. Specifically, VulRepair imposes a maximum limit of 512 code tokens, yet a substantial portion of real-world vulnerable code surpasses this limit. To address these challenges, VulMaster [265] adopts the Fusion-in-Decoder (FiD) framework. It employs a divide-and-conquer approach, and instead of truncating, FiD allows VulMaster to process the entire vulnerable code, regardless of its length. It works by individually encoding multiple input components such as segments of the vulnerable code's token sequence, its AST node sequence, and various forms of CWE expert knowledge. The contextual embeddings from these individually encoded segments are then concatenated into a composite representation, which is subsequently fed into a single decoder to generate the fixed code. Another study [266] also suggests an efficient format for representing code repair modifications. For the input, the framework structures the prompt by clearly identifying the vulnerable line numbers, providing contextual information about the vulnerability type, and including the complete source code of the function with its line numbering. For the output, the format precisely indicates the lines preceding and following the intended code insertion point, with the new code segment clearly marked for straightforward application. The primary advantages of this approach are threefold: It enhances efficiency by generating only new code, thereby reducing tokens and computational overhead; it improves accuracy by clearly delimiting insertion points to prevent misplaced modifications; and it ensures clarity and versatility, making the proposed changes easy to understand, apply, and adaptable to various scenarios. Their experimental results show that, even with a beam size of only 5, the proposed model outperformed VulRepair with a beam size of 50.

The granularity of vulnerabilities within the input code snippet provided for repair is typically identified at the function level or the line level. Models with line-level granularity assume that they know the vulnerable line because they heavily rely on vulnerability repair datasets where precise vulnerability locations are specified at the line level. However, this assumption is largely not applicable in real-world scenarios due to the significant challenges in achieving accurate line-level vulnerability localization and without them, the repair process either lacks the precise target required for an effective fix or becomes heavily reliant on time-consuming and error-prone human efforts to bridge the gap between general detection and specific repair. Therefore, accurate detection and localization are essential prerequisites in vulnerability repair. In this regard, several studies have explored the potentials of multitask learning frameworks, allowing them to be simultaneously trained for both vulnerability detection and repair tasks in this area [267, 268]. In this regard, LineJLocRepair [268] is an end-to-end LLM-based approach for automated software vulnerability localization and repair. It was developed to address the limitations of previous AVR methods, such as their heavy reliance on datasets with already specified precise vulnerability locations and their suboptimal performance in generating long patch sequences. LineJLocRepair tackles these issues through a joint training framework that seamlessly integrates both vulnerability localization and repair tasks, enabling their simultaneous training. This architecture allows the model to accept a vulnerable function as input and directly generate patched code as output. Furthermore, some studies have focused on providing actionable, human-understandable guidance for addressing software vulnerabilities [269, 270].

As shown in Table 13, perfect prediction is the main adopted evaluation metric within AVR models. This metric defines a repair as correct only if the model-generated patch precisely matches the human-written ground-truth in an exact, token-for-token sequence. While it sets a rigorous standard for accuracy, this approach has significant limitations in reflecting real-world utility. It fails to account for functionally correct yet syntactically different solutions, potentially underestimating model performance. The metric's reliance on exact textual matches disregards the presence of multiple valid fixes, making comprehensive dataset construction difficult and potentially penalizing correct but nonidentical repairs [266].

LLM-based methods for vulnerability repair models remain in the emerging stage of the TRL framework due to unresolved challenges in the reliability and correctness of generated patches. Although there is a substantial body of work in this area, these approaches have not yet reached acceptable performance for deployment in operational environments because of issues such as incomplete context understanding, generation of insecure or nonfunctional fixes, and limited validation mechanisms. These limitations highlight the need for more robust evaluation frameworks and tighter integration with formal verification and testing pipelines before broader adoption can be achieved.

Vulnerability repair datasets remain relatively limited in comparison with the more extensive collections available for vulnerability detection tasks. Notable datasets specifically designed for repair include CVEfixes [182], which comprises approximately 11,000 vulnerability-fixing commits [183], and BigVul [167], containing around 217,000 vulnerable and nonvulnerable

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 13 | Overview of vulnerability repair models.

Reference	Name	Methods	LLM	Code	Granularity	Datasets	PP
[264]	VulRepair (2022)	Fine-tuning	CodeT5	C/C++	Line	[167, 182]	44%
[271]	ContractTinker (2023)	Prompt engineering	GPT-3.5	Solidity	Function	Self-built	48%
[259]	SecRepair (2024)	Instruction tuning, reinforcement learning	CodeGen2	C/C++	Function	Self-built	—
[265]	VulMaster (2024)	Continual pretraining, fine-tuning	CodeT5, GPT-3.5	C/C++	Line	[167, 182]	20%
[269]	VulAdvisor (2024)	Prompt engineering, fine-tuning	GPT-3.5, and CodeT5	C/C++	Function	Self-built	—
[266]	— (2024)	Fine-tuning	Code Llama, and Mistral	C/C++	Line	[167, 182]	57%
[267]	DCodeBERT (2025)	Continual pretraining, fine-tuning	CodeBERT	Python, Java, PHP, C++, JavaScript, Ruby	Function	Self-built	—
[268]	LineJLocRepair (2025)	Continual pretraining, fine-tuning	CodeT5	Python, Java, Ruby, JavaScript, Go, PHP, C, C++, C#	Function	[167, 182]	47%

Abbreviation: PP, prefect prediction.

function pairs [168]. These datasets collectively represent a growing but still constrained resource base. Despite increased attention in recent years, datasets in this field frequently suffer from issues such as label noise, data imbalance, duplication, and limited coverage across programming languages and vulnerability types. Furthermore, the lack of comprehensive test cases and validation artifacts for many repair instances poses significant challenges for reliable evaluation, thereby hindering model performance and limiting generalization to real-world scenarios.

7.2 | IR

IR is a structured and well-planned approach that an organization employs to manage the impact of cybersecurity incidents. It primarily maps to the NIST CSF functions Respond (RS) for coordinating containment, mitigation, and communication, and Recover (RC) for restoring systems and services. In the dynamic and fast-paced domain of cloud operations, incident management represents a formidable operational complexity for large-scale cloud service providers such as Microsoft, Google, and Amazon. These incidents may vary in severity, spanning from minor service disruptions to major system outages, and can lead to significant consequences including financial losses, operational downtime, reputational damage, and potential legal liabilities [272]. Consequently, recent research efforts have focused on reducing the duration of this process to enable faster responses to cybersecurity incidents [273–275]. In this regard, in Ref. [273], they propose a framework to investigate LLM-based multiagent collaboration in IR scenarios, employing the Backdoors & Breaches tabletop game as a simulated environment for cybersecurity training. The methodology involves simulating realistic IR dynamics with various team structures. By analyzing the interactions and performance of these LLM-based agents across different setups, the research provides valuable insights into optimizing multiagent collaboration for the IR.

Several studies have also explored the applications of LLMs in the domains of planning [276, 277] and mitigation [272, 278]. Planning involves the development and continuous refinement of structured procedures to ensure timely and coordinated actions during cybersecurity events. A recent study [277] explores the capabilities of LLMs from a reinforcement learning perspective to achieve automated tactics planning. This automation offers the advantages of increased efficiency by reducing response times, enhanced precision in detecting subtle patterns, continuous 24/7 monitoring and response capabilities, and optimized resource allocation by freeing up human personnel. Moreover, mitigation models involve the strategies and actions taken to contain a cyber incident, prevent further damage, and restore normal operations. A recent work [272] addresses the obstacles associated with incident management for enterprise-level cloud service providers. It emphasizes on complex incidents frequently require human intervention, hindered by troubleshooting guides (TSGs) that are often unstructured, inconsistent in quality, and outdated. The Nissist methodology employs a knowledge base to transform unstructured TSGs and mitigation history into structured, executable knowledge nodes, along with a multiagent system that interprets the intents of OCEs, retrieves relevant nodes, and provides systematic, semiautomated actions to expedite incident mitigation. User experiments with OCEs demonstrated that Nissist significantly improved incident mitigation, achieving a time-to-mitigate reduction of 98.93% for

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

simple incidents and 94.85% for hard ones compared to manual mitigation. It also showed a 77.19% full automation success rate for simple incidents and 52.63% for hard ones, with only minor human intervention needed for remaining cases.

There are also some more studies exploring the application of LLMs in log-based recommendations task [64, 279, 280]. This task focuses on automatically generating actionable solutions once anomalous logs have been detected. While the existing log-related methods concentrate on tasks such as compression, parsing, and anomaly detection, engineers still face significant challenges and manual effort in generating actionable resolutions for detected anomalous logs.

Despite the aforementioned advantages of employing LLMs, the volume of published research focusing on their application remains considerably lower compared to other domains within cybersecurity defense. This can be attributed to several significant obstacles. Firstly, the IR field suffers from a severe scarcity of real-world, detailed, and publicly accessible datasets, largely due to privacy and confidentiality concerns, which traditional anonymization methods often fail to adequately address, thereby hindering systematic research and robust model training [281]. Second, the existing log analysis and cybersecurity research have historically prioritized threat detection over automated resolution and repair, leaving a critical gap in generating actionable mitigation steps and requiring substantial manual effort from human experts [280]. Third, IR compounded by the inherent complexity and dynamic nature, which demands rapid decision-making, adaptation to evolving threats, and deep domain knowledge, often proves challenging for conventional pretrained models [282]. Lastly, the absence of comprehensive, standardized benchmarks tailored for evaluating LLM performance across the multiphased and intricate tasks of IR impedes comparable and systematic advancements in this specialized area [277]. Collectively, these obstacles position LLM-based methods within IR at the emerging stage of the TRL framework.

Another observation that we can make in this section is the high adoption of LLM-based agents in this field, as shown in Table 14. This approach enhances flexibility, supports parallel task execution, and allows for dynamic adaptation to evolving threat scenarios. Multiagent architectures further enhance these benefits by mimicking human IR team structures with specialized, collaborative components, allowing for clear divisions of responsibility that mitigate issues such as factual inaccuracy and context loss common in monolithic LLMs [273, 275]. This collective intelligence, combined with their ability to build and refine comprehensive knowledge bases from diverse sources, makes LLM-based multiagent systems a transformative approach for more efficient and effective IR.

Answer of RQ4: Within the three phases of the incident lifecycle, the Response phase has received the least research attention. This is largely due to its complex and mission-critical nature, which necessitates further investigation. Nonetheless, studies such as Refs. [272, 275] have demonstrated models capable of achieving acceptable performance in real-world operations.

8 | RQ5: What Are the Challenges and Future Research Directions for Applying LLMs in SOC Domain?

8.1 | Challenges

Despite the significant advancements and promising potential of LLMs in transforming cybersecurity and SOCs, their application is met with a unique set of challenges and limitations. These hurdles span multiple dimensions, including the models themselves, the data they process, and the operational realities of SOC environments. Addressing these challenges is paramount for the safe, effective, and trustworthy deployment of LLMs in critical security operations. Here are the primary challenges for applying LLMs in the SOC domain:

- Resource intensity and financial barriers: A primary category of limitations in LLM-based approaches stems from the intrinsic characteristics of the models themselves, particularly their substantial computational overhead and resource intensity. Training and fine-tuning large-scale models require significant investment in high-performance hardware and supporting infrastructure, which creates notable financial barriers for many organizations. Moreover, reliance on API-based solutions such as GPT-4 introduces recurring operational expenses that can complicate long-term budget planning and sustainability. These economic and resource constraints collectively limit accessibility and hinder the widespread adoption of LLM-based methods across diverse application settings.
- Hallucination and bias: Hallucination and bias represent the critical challenges for the reliable deployment of LLMs within SOC workflows. Hallucination occurs when a model generates factually incorrect outputs, which in a security

TABLE 14 | Overview of LLM-based incident response models.

Reference	Name	Method	LLM	Downstream tasks
[272]	Nissist (2024)	LLM-based agents	LLaMA-2	Incident mitigation
[276]	— (2024)	Prompt engineering	—	Incident response planning
[279]	LogExpert (2024)	Prompt engineering, fine-tuning	BERT, GPT-4	Log-based recommendation
[278]	ShieldGPT (2024)	Prompt engineering	GPT-4	DDOS mitigation
[273]	AutoBnB (2025)	LLM-based agents	GPT-4o	Incident response
[280]	LogUpdater (2025)	Prompt engineering, fine-tuning	UniXcoder, GPT-4o	Log-based recommendation
[274]	- (2025)	LLM-based agents	—	Incident response
[275]	IRCopilot (2025)	LLM-based agents	—	Incident response
[277]	- (2025)	LLM-based agents, reinforcement learning	Various models	Incident response planning

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

setting can manifest as fabricated threat indicators, false-positive escalations, or misleading RCAs, thereby eroding analyst trust and wasting investigative resources. Bias, on the other hand, stems from imbalanced training data and can lead to the systematic underdetection of certain attack vectors, such as those prevalent in underrepresented environments, or oversensitivity to benign patterns that mimic historically over-represented malicious behavior.

- Inconsistency and opacity: The black box nature of LLMs presents a formidable challenge to explainability and interpretability, as the internal representations and decision pathways remain largely inaccessible to human analysts. This lack of transparency severely undermines trust in high-stakes security environments, where comprehending the rationale behind anomalous detections is essential for an effective IR. From a black box perspective, the absence of visibility into model parameters necessitates exhaustive testing across all possible input combinations, under the assumption that complex interdependencies among values may only be revealed through the systematic examination of every case. Moreover, the phenomenon of catastrophic forgetting introduces additional risk, causing models to lose previously acquired threat intelligence and patch awareness when incrementally exposed to new data [262].

- Attack surface expansion: The integration of LLMs expands the organizational attack surface by introducing new vectors susceptible to adversarial techniques, including prompt injection, data poisoning, and model inversion. Malicious actors can exploit these vulnerabilities to manipulate model behavior, exfiltrate sensitive information, or circumvent existing security countermeasures. This expansion also increases the overall complexity and interconnectivity of system components, thereby creating additional indirect pathways through which attacks may propagate across dependent services and workflows [283].

- Data trilemma: The data-related challenges in developing SOC models can be categorized into three primary areas: data availability, data relevance, and data structure. Issues of availability arise from the lack of high-quality datasets [283]. The sensitive and proprietary nature of cybersecurity data raises significant privacy and security concerns when processed by external LLM services, leading organizations to be hesitant about data sharing. Relevance is affected by the dynamic nature of cyber threats landscape, which leads to rapid concept drift, rendering historical data obsolete [68]. Finally, a fundamental mismatch exists between the text-based design of most LLMs and the multimodal and non-linguistic structure of security data, such as network flows and charts, which complicates effective analysis and pattern recognition [85, 209].

- Technical integrations: Within the SOC environment, LLMs confront significant challenges stemming from operational realities. A primary practical hurdle involves the seamless and effective integration of LLMs with existing, complex SOC technology stacks, which are often heterogeneous and comprise a variety of legacy and modern systems. This integration complexity is further compounded by the lack of standardized APIs and data schemas across different security tools, necessitating custom middleware or adapter layers [284].

- Regulatory compliance: The deployment of LLM-based solutions adds another layer of difficulty due to the need to ensure regulatory compliance with critical standards, especially when sensitive user information is processed by external, third-party LLM services via public APIs, raising concerns about data leakage and unauthorized access. For instance, compliance frameworks such as the General Data Protection Regulation (GDPR) and the Health Insurance Portability and Accountability Act (HIPAA) impose strict requirements on data residency, audit logging, and processing transparency, which may conflict with the inherently opaque and cross-jurisdictional nature of commercial LLM APIs.

- Balancing human expertise with automation: There is an ongoing need to find the optimal balance between human expertise and LLM-driven automation. While LLMs can augment human capabilities and automate routine tasks, critical SOC-related decisions often require nuanced human judgment and cannot be fully delegated to autonomous AI agents. This tension is exacerbated by the fact that LLM outputs are probabilistic and context-sensitive, making it difficult to establish deterministic thresholds for when automation should defer to human analysts. The lack of structured processes and heavy reliance on tacit knowledge among analysts further complicate efforts to define clear objectives and measure performance improvements [285].

8.2 | Future Directions

The application of LLMs within SOCs is a rapidly evolving area, holding significant promise for advancing its capabilities. While LLMs have demonstrated remarkable capabilities in NLP tasks and are increasingly being explored for various security functions, their widespread adoption in SOC environments necessitates further research to address existing limitations and unlock their full potential. Future research directions are focused on enhancing their accuracy, efficiency, and interpretability, as well as enabling more autonomous and integrated security operations. Here are key future research directions for applying LLMs in the SOC domain:

- Advancing Explainability and Trust in LLM-Driven SOCs: To alleviate the black box nature of LLMs, future research must focus on developing robust Explainable AI (XAI) techniques specifically tailored for LLMs in cybersecurity contexts. This includes methods to clearly articulate why an LLM classified an alert in a certain way, generated a specific recommendation, or identified a particular threat indicator. Techniques such as attention visualization, counterfactual explanations, and natural language justifications need further refinement to be practically useful in high-pressure SOC environments. Building trust also involves creating mechanisms for verifiability and human oversight, allowing analysts to easily validate LLM outputs and intervene when necessary [8, 136]. Practitioners should start by implementing post hoc explanation tools (e.g., LIME or SHAP) on any LLM-based detection system and establish a mandatory human review process for any alert where the explanation confidence falls below a predefined threshold. Building trust

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

also involves creating mechanisms for verifiability and human oversight, allowing analysts to easily validate LLM outputs and intervene when necessary.

- Integration of LLMs and Federated Learning: The integration of LLMs with FL is a promising future direction, particularly for enhancing security in distributed and privacy-sensitive environments. This decentralized paradigm significantly enhances scalability and fault tolerance for systems, enabling organizations to share CTI and collectively build more robust defense mechanisms [286]. Additionally, this collaborative sharing of cybersecurity information is vital for vulnerability management, threat detection, real-time monitoring, and IR across IoT ecosystems [287, 288]. While traditional FL-based anomaly detection frameworks often rely on manually designed numerical features, limiting their ability to grasp contextual and semantic patterns, LLMs can provide this crucial contextual understanding, thereby improving robustness against adversarial manipulations [98]. For practitioners operating multitenant cloud SOCs, a recommended actionable step is to pilot FL with a small consortium of trusted partners using open-source LLMs before scaling to larger deployments, while ensuring differential privacy guarantees are embedded at the client level.
- Robust Benchmarks and Datasets: A significant challenge for LLM applications in SOCs is the lack of standardized benchmarks and comprehensive evaluation methodologies. Future research needs to focus on creating more robust and representative datasets and benchmarks to accurately assess LLMs' performance in diverse SOC tasks [47]. Moreover, there is a particular demand for simulation environments that can realistically mimic SOC operations and allow for the evaluation of LLM decision-making in different threat scenarios [245]. These environments can help address data quality issues, mitigate biases introduced by existing datasets, and ensure that models can adapt to evolving threat landscapes and previously unseen attacks [288]. Practitioners should adopt a continuous benchmarking pipeline using freely available simulation toolkits (e.g., SOCSim, DARPA TC) and regularly re-evaluate models against a held-out set of recent incident reports to detect performance decay.
- Agent-Based RAG: Future research will increasingly focus on agent-based RAG systems, where LLM agents are endowed with the ability to dynamically query and integrate knowledge from various sources to perform complex tasks. These agents, capable of iterative interaction with their environment, reasoning about tasks, and planning actions, can significantly enhance functions like RCA by accessing diverse information sources [254]. By combining the retrieval capabilities of RAG with the autonomous decision-making and tool-use of agents, such systems can provide more accurate, context-rich, and verifiable outputs for security professionals [289]. A concrete guideline for SOC engineers is to implement a simple agent-based RAG prototype for a single high-value use case (e.g., alert enrichment) with explicit retrieval-source logging and then measure the reduction in mean time to respond (MTTR) before broadening deployment.
- Autonomous Security Decision-Making: The future of SOCs involves a significant shift toward increased automation and orchestration, moving LLMs beyond mere assistant roles to more autonomous agents capable of handling complex decision-making tasks under human supervision. This entails developing LLMs that can autonomously perform tasks such as automated patch management, antivirus management, network reconfiguration, and port management [79]. The goal is to streamline workflows and minimize manual intervention, enhancing SOC efficiency and enabling faster decision-making in response to dynamic cyber threats [290]. Practitioners should begin with low-risk, reversible actions (e.g., automated ticket prioritization or recommended patch lists) and require human approval for any critical actions (e.g., network-altering or data-destructive) until the system demonstrates a sustained track record of safe decisions.
- Hybrid Architectures: Hybrid models leverage the strengths of different techniques to overcome individual limitations, leading to more lightweight, compact, and real-time accountable systems. This could involve traditional rule-based systems with LLMs, where LLMs provide contextual understanding and adapt rules to evolving threats. Rule-based systems ensure precision and deterministic outputs in known scenarios. Alternatively, integrating ML techniques with LLMs can also create another powerful hybrid model. Such hybrid approaches aim to achieve high accuracy and explainability while optimizing computational resources for real-time deployment. A practical best practice is to deploy a two-stage pipeline: a lightweight rule-based filter for high-certainty known indicators, followed by an LLM only for borderline cases, thereby reducing API costs and latency while maintaining accuracy.
- Establish a Human-in-the-Loop (HITL) Protocol: Future research must prioritize defining robust HITL protocols to govern the interaction between analysts and LLM-powered systems. A critical component of this protocol is establishing clear escalation paths for instances where the LLM's predicted confidence level is low or when a human analyst chooses to over-ride an automated recommendation. These analyst over-rides represent crucial, high-value feedback signals that must be systematically captured. This formalized feedback loop is essential for the continuous improvement of the underlying models, ensuring that the SOC functions as a learning organization by continuously refining accuracy and increasing decision transparency [291]. Practitioners should immediately implement a version-controlled feedback registry that logs every analyst override together with the input context and the LLM's original output, and schedule monthly fine-tuning sessions using this accumulated feedback.
- Filling Gaps in the NIST CSF: Our survey shows a heavy concentration of research on the "Detect" function, with growing work in "Respond." However, there is a significant research gap in studies addressing the Protect and Recover functions. Future research should investigate how LLMs can proactively enhance defense mechanisms by generating stronger security configurations, automating vulnerability patch prioritization, and simulating attack scenarios to assess system resilience. Additionally, the Recover function

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

requires greater focus, particularly on employing LLMs to automate system restoration, verify backup data integrity, and facilitate comprehensive postincident analysis to prevent recurrence. As an actionable guideline, SOC managers should run quarterly tabletop exercises that explicitly task LLM-augmented teams with developing a recovery plan from a given compromise scenario, measuring the completeness and correctness of generated steps against a checklist of NIST Recovery controls.

Answer of RQ5: The most critical challenges facing the proposed models in this domain include hallucinations, adversarial attacks against LLMs, the absence of standardized benchmarks, and regulatory compliance constraints inherent to SOC tasks. Furthermore, integrating LLMs with advanced and emerging technologies such as federated learning, as well as combining it with traditional rule-based or ML-based approaches, represents a promising direction with significant potential for future research.

9 | Conclusion

The escalating sophistication and frequency of cyber threats have placed unprecedented strain on SOCs, exacerbating challenges such as alert fatigue, a critical skills gap, and delayed IR. This survey has systematically explored the transformative potential of LLMs to address these pressing operational realities. By providing the first comprehensive, structured analysis of LLM applications within the SOC workflow, this paper offers a valuable resource for researchers, SOC managers, and cybersecurity professionals. Our key findings reveal a rapidly evolving landscape. We observed a clear preference for BERT-based models in the Detection phase, often fine-tuned for high-accuracy classification tasks, and a strong adoption of GPT-based models in the Analysis and Response phases, where generative capabilities and in-context learning are paramount. By mapping these applications to the NIST CSF and using the MITRE ATT&CK framework as an analytical lens, we demonstrated that current research is heavily concentrated on the NIST "Detect" function, with significant gaps remaining in the "Protect" and "Recover" functions. The primary contributions of this work are threefold:

1. A foundational overview: we provided a clear and structured explanation of SOC operations, analyst tiers, and the challenges they face in the modern threat landscape.
2. A structured analysis: we introduced a novel analytical framework that maps LLM applications to the authoritative NIST CSF and MITRE ATT&CK standards. This approach provides a rigorous and replicable method for evaluating and understanding the role of AI in security operations.
3. A research roadmap: we identified critical research gaps and proposed concrete future directions, guiding the community toward developing more autonomous, secure, and comprehensive AI-driven defenses that span the entire incident lifecycle.

To translate these findings into tangible improvements for SOC operations, practitioners are advised to adopt the following evidence-based guidelines. First, prioritize deployment of fine-tuned BERT-based models for detection tasks where high precision and low latency are critical, while reserving generative GPT-based models for investigative and response phases that benefit from contextual summarization and narrative generation. Second, establish a phased integration strategy: Begin with nonoperational, read-only use cases to build confidence and measure baseline performance before progressing to semi-automated actions. Third, implement continuous evaluation loops using frameworks such as RAGAS or custom-built metrics that track answer relevance, faithfulness, and alignment with MITRE ATT&CK mappings; any model exhibiting output inconsistency or hallucination above a predefined threshold should be recalibrated or routed to human analysts. Fourth, for privacy-sensitive environments, prefer on-premises or sovereign cloud deployments of open-source LLMs over API-based third-party services to mitigate data leakage risks. Finally, SOC managers should adopt structured playbooks that delineate human-only decisions from LLM-augmented workflows, while mandating a HITL approach to verify and refine all model outputs. This practice acknowledges that current LLMs lack the experiential depth, contextual judgment, and interactive capabilities of human analysts and therefore serve as complementary tools rather than autonomous replacements. Adhering to these principles will help organizations move beyond isolated pilot studies toward robust, reproducible, and risk-aware integration of LLMs into daily security operations.

Author Contributions

Ali Habibzadeh: conceptualization, methodology, visualization, and writing–original draft preparation. Farid Feyzi: conceptualization, writing–review and editing, and supervision. Reza Ebrahimi Atani: writing–review and editing and supervision.

Funding

The authors declare that no funding was received from any organization or agency in support of this research.

Disclosure

A preprint has previously been published [292].

Ethics Statement

This article does not contain any studies with animals performed by any of the authors.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

Data sharing is not applicable to this article as no datasets were generated or analyzed during the current study.

References

1. S. Minaee, T. Mikolov, N. Nikzad, et al., "Large Language Models: A Survey," (2024), https://arxiv.org/abs/2402.06196.

2. H. Zhou, C. Hu, Y. Yuan, et al., "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities," *IEEE Communications Surveys & Tutorials* 27, no. 3 (2024): 1955–2005, https://doi.org/10.1109/COMST.2024.3465447.

3. R. Gong and X. Li, "The Application Progress and Research Trends of Knowledge Graphs and Large Language Models in Agriculture,"

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Computers and Electronics in Agriculture 235 (2025): 110396, https://doi.org/10.1016/j.compag.2025.110396.

4. M. M. Karim, Y. Shi, S. Zhang, B. Wang, M. Nasri, and Y. Wang, "Large Language Models and Their Applications in Roadway Safety and Mobility Enhancement: A Comprehensive Review," *Artificial Intelligence for Transportation* 1 (2025): 100004, https://doi.org/10.1016/j.ait.2025.100004.

5. Z. Wang, H. Li, D. Huang, H.-S. Kim, C.-W. Shin, and A. M. Rahmani, "HealthQ: Unveiling Questioning Capabilities of LLM Chains in Healthcare Conversations," *Smart Health* 36 (2025): 100570, https://doi.org/10.1016/j.smhl.2025.100570.

6. M. Hassanin and N. Moustafa, "A Comprehensive Overview of Large Language Models (LLMs) for Cyber Defences: Opportunities and Directions," (2024), https://arxiv.org/abs/2405.14487.

7. B. Abdeen, E. Al-Shaer, A. Singhal, L. Khan, and K. Hamlen, "SMET: Semantic Mapping of CVE to ATT&CK and Its Application to Cybersecurity," in *Proceedings of the 2023 IFIP Annual Conference on Data and Applications Security and Privacy* (Sophia-Antipolis, 2023), 243–260, https://doi.org/10.1007/978-3-031-37586-6_15.

8. M. Khayat, E. Barka, M. A. Serhani, F. Sallabi, K. Shuaib, and H. M. Khater, "Empowering Security Operation Center With Artificial Intelligence and Machine Learning–A Systematic Literature Review," *IEEE Access* 13 (2025): 19162–19197, https://doi.org/10.1109/ACCESS.2025.3532951.

9. C. Crowley and The SANS Institute, "SANS 2025 SOC Survey: State of the SOC Survey," (2025), https://www.sans.org/white-papers/sans-2025-soc-survey/.

10. U.S. Bureau of Labor Statistics, Information Security Analysts, U.S. Bureau of Labor Statistics, "Occupational Outlook Handbook," (2025), https://www.bls.gov/ooh/computer-and-information-technology/information-security-analysts.htm.

11. Ponemon Institute, "2024 Cost of a Data Breach Report," (2024), https://www.ibm.com/reports/data-breach.

12. Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A Survey on Large Language Model (LLM) Security and Privacy: The Good, the Bad, and the Ugly," *High-Confidence Computing* 4, no. 2 (2024): 100211, https://doi.org/10.1016/j.hcc.2024.100211.

13. J. Zhang, H. Bu, H. Wen, et al., "When LLMs Meet Cybersecurity: A Systematic Literature Review," *Cybersecurity* 8, no. 1 (2025): 55, https://doi.org/10.1186/s42400-025-00361-w.

14. S. M. Taghavi Far and F. Feyzi, "Large Language Models for Software Vulnerability Detection: A Guide for Researchers on Models, Methods, Techniques, Datasets, and Metrics," *International Journal of Information Security* 24, no. 2 (2025): 78, https://doi.org/10.1007/s10207-025-00992-7.

15. K. Tamberg and H. Bahsi, "Harnessing Large Language Models for Software Vulnerability Detection: A Comprehensive Benchmarking Study," *IEEE Access* 13 (2025): 29698–29717, https://doi.org/10.1109/ACCESS.2025.3541146.

16. X. Zhou, S. Cao, X. Sun, and D. Lo, "Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead," *ACM Transactions on Software Engineering and Methodology* 34, no. 5 (2025): 1–31, https://doi.org/10.1145/3708522.

17. Y. Chen, M. Cui, D. Wang, et al., "A Survey of Large Language Models for Cyber Threat Detection," *Computers & Security* 145 (2024): 104016, https://doi.org/10.1016/j.cose.2024.104016.

18. C. Feng, S. Wu, and N. Liu, "A User-Centric Machine Learning Framework for Cyber Security Operations Center," in *Proceedings of the 2017 IEEE International Conference on Intelligence and Security Informatics (ISI)* (Beijing, 2017), 173–175, https://doi.org/10.1109/ISI.2017.8004902.

19. S. Oesch, R. Bridges, J. Smith, et al., "An Assessment of the Usability of Machine Learning Based Tools for the Security Operations Center," in *Proceedings of the 2020 International Conferences on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics (Cybermatics)* (Rhodes, 2020), 634–641.

20. Y. Chang, X. Wang, J. Wang, et al., "A Survey on Evaluation of Large Language Models," *ACM Transactions on Intelligent Systems and Technology* 15, no. 3 (2024): 1–45, https://doi.org/10.1145/3641289.

21. J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A Survey on Large Language Models for Code Generation," (2024), https://arxiv.org/abs/2406.00515.

22. K. Peng, L. Ding, Q. Zhong, et al., "Towards Making the Most of ChatGPT for Machine Translation," (2023), https://arxiv.org/abs/2303.13780.

23. T. Kocmi and C. Federmann, "Large Language Models Are State-of-the-Art Evaluators of Translation Quality," (2023), https://arxiv.org/abs/2302.14520.

24. L. Wang, W. Bi, S. Zhao, et al., "Investigating the Impact of Prompt Engineering on the Performance of Large Language Models for Standardizing Obstetric Diagnosis Text: Comparative Study," *JMIR Formative Research* 8, no. 1 (2024): e53216, https://doi.org/10.2196/53216.

25. G. Yao, W. Zhang, Y. Zhu, et al., "Comparing the Accuracy of Large Language Models and Prompt Engineering in Diagnosing Realworld Cases," *International Journal of Medical Informatics* 203 (2025): 106026, https://doi.org/10.1016/j.ijmedinf.2025.106026.

26. M. Zhang, Y. Pan, Q. Cui, Y. Lü, and W. Yu, "Multimodal LLM for Enhanced Alzheimer's Disease Diagnosis: Interpretable Feature Extraction From Mini-Mental State Examination Data," *Experimental Gerontology* 208 (2025): 112812, https://doi.org/10.1016/j.exger.2025.112812.

27. M. Cascella, J. Montomoli, V. Bellini, and E. Bignami, "Evaluating the Feasibility of Chatgpt in Healthcare: An Analysis of Multiple Clinical and Research Scenarios," *Journal of Medical Systems* 47, no. 1 (2023): 33, https://doi.org/10.1007/s10916-023-01925-4.

28. J.-w. Lee, I.-S. Yoo, J.-H. Kim, et al., "Development of AI-Generated Medical Responses Using the ChatGPT for Cancer Patients," *Computer Methods and Programs in Biomedicine* 254 (2024): 108302, https://doi.org/10.1016/j.cmpb.2024.108302.

29. A. A. Onitilo, A. R. Shour, D. S. Puthoff, Y. Tanimu, A. Joseph, and M. T. Sheehan, "Evaluating the Adoption of Voice Recognition Technology for Real-Time Dictation in a Rural Healthcare System: A Retrospective Analysis of Dragon Medical One," *PLoS One* 18, no. 3 (2023): e0272545, https://doi.org/10.1371/journal.pone.0272545.

30. Y.-P. Hsiao, N. Klijn, and M.-S. Chiu, "Developing a Framework to Re-Design Writing Assignment Assessment for the Era of Large Language Models," *Learning: Research and Practice* 9, no. 2 (2023): 148–158, https://doi.org/10.1080/23735082.2023.2257234.

31. K. Guo and D. Wang, "To Resist It or to Embrace it? Examining ChatGPT's Potential to Support Teacher Feedback in EFL Writing," *Education and Information Technologies* 29, no. 7 (2024): 8435–8463, https://doi.org/10.1007/s10639-023-12146-0.

32. C. Chhetri, "Exploring Large Language Model-Powered Pedagogical Approaches to Cybersecurity Education," in *Proceedings of the 25th Annual Conference on Information Technology Education* (El Paso, TX, 2024), 163–166, https://doi.org/10.1145/3686852.3686887.

33. S. Elkins, E. Kochmar, I. Serban, and J. C. Cheung, "How Useful Are Educational Questions Generated by Large Language Models?" in *Proceedings of the 2023 International Conference on Artificial Intelligence in Education* (Tokyo, 2023), 536–542, https://doi.org/10.1007/978-3-031-36336-8_83.

34. J. Han, H. Yoo, J. Myung, et al., "LLM-as-a-tutor in EFL Writing Education: Focusing on Evaluation of Student-LLM Interaction," (2023), https://arxiv.org/abs/2310.05191.

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

35. W. Dai, J. Lin, H. Jin, et al., "Can Large Language Models Provide Feedback to Students? A Case Study on ChatGPT," in *Proceedings of the 2023 IEEE International Conference on Advanced Learning Technologies (ICALT)* (Orem, UT, 2023), 323–325, https://doi.org/10.1109/ICALT58122.2023.00100.

36. C. Nelson, A. Doupé, and Y. Shoshitaishvili, "SENSAI: Large Language Models as Applied Cybersecurity Tutors," in *Proceedings of the 56th ACM Technical Symposium on Computer Science Education* (Pittsburgh, PA, 2025), 833–839, https://doi.org/10.1145/3641554.3701801.

37. Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, "No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT," *IEEE Transactions on Software Engineering* 50, no. 6 (2024): 1548–1584, https://doi.org/10.1109/TSE.2024.3392499.

38. Q. Zheng, X. Xia, X. Zou, et al., "CodeGeeX: A Pre-Trained Model for Code Generation With Multilingual Benchmarking on Humaneval-X," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, 2023), 5673–5684, https://doi.org/10.1145/3580305.3599790.

39. M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation," *IEEE Transactions on Software Engineering* 50, no. 1 (2023): 85–105, https://doi.org/10.1109/TSE.2023.3334955.

40. S. Zhao, A. Zhu, H. Mozannar, D. Sontag, A. Talwalkar, and V. Chen, "CodingGenie: A Proactive LLM-Powered Programming Assistant," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Trondheim, 2025), 1168–1172, https://doi.org/10.1145/3696630.3728603.

41. B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "On Hardware Security Bug Code Fixes by Prompting Large Language Models," *IEEE Transactions on Information Forensics and Security* 19 (2024): 4043–4057, https://doi.org/10.1109/TIFS.2024.3374558.

42. I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," (2024), https://arxiv.org/abs/2403.17134.

43. W. Luo, J. Keung, B. Yang, et al., "When Fine-Tuning LLMs Meets Data Privacy: An Empirical Study of Federated Learning in LLM-Based Program Repair," *ACM Transactions on Software Engineering and Methodology* 35, no. 3 (2024): 1–46, https://doi.org/10.1145/3733599.

44. M. Vielberth, "Security Operations Center (SOC)," in *Encyclopedia of Cryptography, Security and Privacy* (Berlin: Springer, 2021), 1–3.

45. C. DeCusatis, R. Cannistra, A. Labouseur, and M. Johnson, "Design and Implementation of a Research and Education Cybersecurity Operations Center," in *Cybersecurity and Secure Information Systems: Challenges and Solutions in Smart Environments* (Berlin: Springer, 2019), 287–310, https://doi.org/10.1007/978-3-030-16837-7_13.

46. M. Vielberth, F. Böhm, I. Fichtinger, and G. Pernul, "Security Operations Center: A Systematic Study and Open Challenges," *IEEE Access* 8 (2020): 227756–227779, https://doi.org/10.1109/ACCESS.2020.3045514.

47. O. Oniagbi, A. Hakkala, and I. Hasanov, *Evaluation of LLM Agents for the SOC Tier 1 Analyst Triage Process* (University of Turku, 2024).

48. N. Gupta, I. Traore, and P. M. F. de Quinan, "Automated Event Prioritization for Security Operation Center Using Deep Learning," in *Proceedings of the 2019 IEEE International Conference on Big Data (Big Data)* (Los Angeles, CA, 2019), 5864–5872, https://doi.org/10.1109/BigData47090.2019.9006073.

49. A. Waleed, A. F. Jamali, and A. Masood, "Which Open-Source IDS? Snort, Suricata or Zeek," *Computer Networks* 213 (2022): 109116, https://doi.org/10.1016/j.comnet.2022.109116.

50. S. Griffith and T. H. Morris, "Network Intrusion Detection System," in *Encyclopedia of Cryptography, Security and Privacy* (Berlin: Springer, 2024), 1–3.

51. National Institute of Standards and Technology, "Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1, Tech. Rep. NIST Cybersecurity Framework," (2018), https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.04162018.pdf.

52. F. Liu, B. Farkiani, and P. Crowley, "Large Language Models for Computer Networking Operations and Management: A Survey on Applications, Key Techniques, and Opportunities," *Computer Networks* 271 (2025): 111614, https://doi.org/10.1016/j.comnet.2025.111614.

53. B. Kitchenham and S. Charters, "Guidelines for Performing Systematic Literature Reviews in Software Engineering," *Empirical Software Engineering* (Berlin: Springer, 2007).

54. NASA, "Technology Readiness Level (TRL) Definitions," (2012), https://www.nasa.gov/directorates/heo/scan/engineering/technology/technology_readiness_level.

55. ISO, "Space Systems – Definition of the Technology Readiness Levels (TRLs) and Their Criteria of Assessment, Tech. Rep. ISO 16290:2013," (2013), https://www.iso.org/standard/56064.html.

56. H. Guo, S. Yuan, and X. Wu, "LogBERT: Log Anomaly Detection via BERT," in *Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN)* (Shenzhen, 2021), 1–8, https://doi.org/10.1109/IJCNN52387.2021.9534113.

57. X. Han, S. Yuan, and M. Trabelsi, "LogGPT: Log Anomaly Detection via GPT," in *Proceedings of the 2023 IEEE International Conference on Big Data (BigData)* (Sorrento: IEEE, 2023), 1117–1122, https://doi.org/10.1109/BigData59044.2023.10386543.

58. Y. Lee, J. Kim, and P. Kang, "LAnoBERT: System Log Anomaly Detection Based on BERT Masked Language Model," *Applied Soft Computing* 146 (2023): 110689, https://doi.org/10.1016/j.asoc.2023.110689.

59. C. Almodovar, F. Sabrina, S. Karimi, and S. Azad, "LogFiT: Log Anomaly Detection Using Fine-Tuned Language Models," *IEEE Transactions on Network and Service Management* 21, no. 2 (2024): 1715–1723, https://doi.org/10.1109/TNSM.2024.3358730.

60. V.-H. Le and H. Zhang, "Log-Based Anomaly Detection Without Log Parsing," in *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (Melbourne, 2021), 492–504, https://doi.org/10.1109/ASE51524.2021.9678773.

61. S. Chen and H. Liao, "BERT-Log: Anomaly Detection for System Logs Based on Pre-trained Language Model," *Applied Artificial Intelligence* 36, no. 1 (2022): 2145642, https://doi.org/10.1080/08839514.2022.2145642.

62. L. A. Jilcha, D.-H. Kim, and J. Kwak, "SaRLog: Semantic-Aware Robust Log Anomaly Detection via BERT-Augmented Contrastive Learning," *IEEE Internet of Things Journal* 11, no. 13 (2024): 23727–-23736, https://doi.org/10.1109/JIOT.2024.3386183.

63. W. Guan, J. Cao, S. Qian, J. Gao, and C. Ouyang, "LogLLM: Log-Based Anomaly Detection Using Large Language Models," (2024), https://arxiv.org/abs/2411.08561.

64. Y. Liu, Y. Ji, S. Tao, et al., "LogLM: From Task-Based to Instruction-Based Automated Log Analysis," in *Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (Ottawa, 2025), 401–412, https://doi.org/10.1109/ICSE-SEIP66354.2025.00041.

65. L. Ma, W. Yang, S. Jiang, et al., "LUK: Empowering Log Understanding With Expert Knowledge From Large Language Models," *IEEE Transactions on Software Engineering* 51, no. 10 (2025): 2764–2786, https://doi.org/10.1109/TSE.2025.3594046.

66. Z. Zhang, S. Li, L. Zhang, J. Ye, C. Hu, and L. Yan, "LLM-LADE: Large Language Model-Based Log Anomaly Detection With Explanation," *Knowledge-Based Systems* 326 (2025): 114064, https://doi.org/10.1016/j.knosys.2025.114064.

67. J. Qi, S. Huang, Z. Luan, et al., "LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection," in *Proceedings of the 2023 IEEE International*

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys) (Melbourne, 2023), 273–280, https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys60770.2023.00045.

68. Y. Liu, S. Tao, W. Meng, et al., "Interpretable Online Log Analysis Using Large Language Models With Prompt Strategies," in *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension* (Lisbon, 2024), 35–46, https://doi.org/10.1145/3643916.3644408.

69. C. Song, L. Ma, J. Zheng, J. Liao, H. Kuang, and L. Yang, "Audit-LLM: Multi-Agent Collaboration for Log-Based Insider Threat Detection," (2024), https://arxiv.org/abs/2408.08902.

70. L. Huang, W. Yu, W. Ma, et al., "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions," *ACM Transactions on Information Systems* 43, no. 2 (2025): 1–55, https://doi.org/10.1145/3703155.

71. J. Pan, W. S. Liang, and Y. Yidi, "RAGLog: Log Anomaly Detection Using Retrieval Augmented Generation," in *Proceedings of the 2024 IEEE World Forum on Public Safety Technology (WFPST)* (Washington, DC, 2024), 169–174, https://doi.org/10.1109/WFPST58552.2024.00034.

72. G. No, Y. Lee, H. Kang, and P. Kang, "Training-Free Retrieval-Based Log Anomaly Detection With Pre-Trained Language Model Considering Token-Level Information," *Engineering Applications of Artificial Intelligence* 133 (2024): 108613, https://doi.org/10.1016/j.engappai.2024.108613.

73. J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A Large Collection of System Log Datasets for AI-Driven Log Analytics," in *Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)* (Florence, 2023), 355–366, https://doi.org/10.1109/ISSRE59848.2023.00071.

74. D. C. Le, N. Zincir-Heywood, and M. I. Heywood, "Analyzing Data Granularity Levels for Insider Threat Detection Using Machine Learning," *IEEE Transactions on Network and Service Management* 17, no. 1 (2020): 30–44, https://doi.org/10.1109/TNSM.2020.2967721.

75. C. Laprade, B. Bowman, and H. H. Huang, "PicoDomain: A Compact High-Fidelity Cybersecurity Dataset," (2020), https://arxiv.org/abs/2008.09192.

76. A. Oliner and J. Stearley, "What Supercomputers Say: A Study of Five System Logs," in *Proceedings of the 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)* (Edinburgh, 2007), 575–584, https://doi.org/10.1109/DSN.2007.103.

77. P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," in *Proceedings of the 2017 IEEE International Conference on Web Services (ICWS)* (Honolulu, HI, 2017), 33–40, https://doi.org/10.1109/ICWS.2017.13.

78. P. R. Houssel, P. Singh, S. Layeghy, and M. Portmann, "Towards Explainable Network Intrusion Detection Using Large Language Models," (2024), https://arxiv.org/abs/2408.04342.

79. M. A. Ferrag, M. Ndhlovu, N. Tihanyi, et al., "Revolutionizing Cyber Threat Detection With Large Language Models: A Privacy-Preserving BERT-Based Lightweight Model for IoT/IIoT Devices," *IEEE Access* 12 (2024): 23733–23750, https://doi.org/10.1109/ACCESS.2024.3363469.

80. M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning," *IEEE Access* 10 (2022): 40281–40306, https://doi.org/10.1109/ACCESS.2022.3165809.

81. O. G. Lira, A. Marroquin, and M. A. To, "Harnessing the Advanced Capabilities of LLM for Adaptive Intrusion Detection Systems," in *Proceedings of the 2024 International Conference on Advanced Information Networking and Applications* (Kitakyushu, 2024), 453–464, https://doi.org/10.1007/978-3-031-57942-4_44.

82. M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," in *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications* (Ottawa, 2009), 1–6.

83. A. Diaf, A. A. Korba, N. E. Karabadji, and Y. Ghamri-Doudane, "Beyond Detection: Leveraging Large Language Models for Cyber Attack Prediction in IoT Networks," in *Proceedings of the 2024 20th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)* (Abu Dhabi, 2024), 117–123, https://doi.org/10.1109/DCOSS-IoT61029.2024.00026.

84. E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," *Sensors* 23, no. 13 (2023): 5941, https://doi.org/10.3390/s23135941.

85. Q. Li, Y. Zhang, Z. Jia, et al., "DoLLM: How Large Language Models Understanding Network Flow Data to Detect Carpet Bombing DDoS," (2024), https://arxiv.org/abs/2405.07638.

86. I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy," in *Proceedings of the 2019 International Carnahan Conference on Security Technology (ICCST)* (Chennai, 2019), 1–8, https://doi.org/10.1109/CCST.2019.8888419.

87. M. Fu, P. Wang, M. Liu, Z. Zhang, and X. Zhou, "IoV-BERT-IDS: Hybrid Network Intrusion Detection System in IoV Using Large Language Models," *IEEE Transactions on Vehicular Technology* 74, no. 2 (2024): 1909–1921, https://doi.org/10.1109/TVT.2024.3402366.

88. I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, et al., "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy* (Funchal, 2018), 108–116.

89. N. Koroniotis, N. Moustafa, and E. Sitnikova, "A New Network Forensic Framework Based on Deep Learning for Internet of Things Networks: A Particle Deep Framework," *Future Generation Computer Systems* 110 (2020): 91–106, https://doi.org/10.1016/j.future.2020.03.042.

90. H. M. Song, J. Woo, and H. K. Kim, "In-Vehicle Network Intrusion Detection Using Deep Convolutional Neural Network," *Vehicular Communications* 21 (2020): 100198, https://doi.org/10.1016/j.vehcom.2019.100198.

91. Y. Li, Z. Xiang, N. D. Bastian, D. Song, and B. Li, *IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks* (NIPS, 2024).

92. E. A. Nack, M. C. McKenzie, and N. D. Bastian, "ACI-IoT-2023: A Robust Dataset for Internet of Things Network Security Analysis," in *Proceedings of the MILCOM 2024-2024 IEEE Military Communications Conference (MILCOM)* (Washington, DC, 2024), 1–6, https://doi.org/10.1109/MILCOM61039.2024.10773916.

93. H. Zhang, A. B. Sediq, A. Afana, and M. Erol-Kantarci, "Large Language Models in Wireless Application Design: In-Context Learning-Enhanced Automatic Network Intrusion Detection," (2024), https://arxiv.org/abs/2405.11002.

94. B. Farzaneh, N. Shahriar, A. H. Al Muktadir, and M. S. Towhid, "DTL-IDS: Deep Transfer Learning-Based Intrusion Detection System in 5G Networks," in *Proceedings of the 2023 19th International Conference on Network and Service Management (CNSM)* (Niagara Falls, 2023), 1–5, https://doi.org/10.23919/CNSM59352.2023.10327918.

95. A. Diaf, A. A. Korba, N. Elislem Karabadji, and Y. Ghamri-Doudane, "BARTPredict: Empowering IoT Security With LLM-Driven Cyber Threat Prediction," in *Proceedings of the GLOBECOM 2024 – 2024 IEEE Global Communications Conference* (2024), 1239–1244, https://doi.org/10.1109/GLOBECOM52923.2024.10901770.

96. X. Li, Z. Zheng, M. Zhao, Y. Zhao, L. Shi, and B. Wang, "RLFE-IDS: A Framework of Intrusion Detection System Based on Retrieval Augmented Generation and Large Language Model," *Computer Networks* 268 (2025): 111341, https://doi.org/10.1016/j.comnet.2025.111341.

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

97. Canadian Institute for Cybersecurity, "CICIDS2018 Dataset," (2018), https://www.unb.ca/cic/datasets/ids-2018.html.

98. H. Rezaei, R. Taheri, and M. Shojafar, "FedLLMGuard: A Federated Large Language Model for Anomaly Detection in 5G Networks," *Computer Networks* 269 (2025): 111473, https://doi.org/10.1016/j.comnet.2025.111473.

99. D. Herzalla, W. T. Lunardi, and M. Andreoni, "TII-SSRC-23 Dataset: Typological Exploration of Diverse Traffic Patterns for Intrusion Detection," *IEEE Access* 11 (2023): 118577–118594, https://doi.org/10.1109/ACCESS.2023.3319213.

100. M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "Netflow Datasets for Machine Learning-Based Network Intrusion Detection Systems," in *Proceedings of the 2020 International Conference on Big Data Technologies and Applications* (Springer, 2020), 117–135, https://doi.org/10.1007/978-3-030-72802-1_9.

101. Z. Wang, J. Li, S. Yang, X. Luo, D. Li, and S. Mahmoodi, "A Lightweight IoT Intrusion Detection Model Based on Improved BERT-of-Theseus," *Expert Systems With Applications* 238 (2024): 122045, https://doi.org/10.1016/j.eswa.2023.122045.

102. A. Belenguer, J. A. Pascual, and J. Navaridas, "A Review of Federated Learning Applications in Intrusion Detection Systems," *Computer Networks* 258 (2025): 111023, https://doi.org/10.1016/j.comnet.2024.111023.

103. R. Beuran and R. Beuran, "FedMSE: Semi-Supervised Federated Learning Approach for IoT Network Intrusion Detection," *Computers & Security* 151 (2025): 104337, https://doi.org/10.1016/j.cose.2025.104337.

104. F. Adjewa, M. Esseghir, and L. Merghem-Boulahia, "Efficient Federated Intrusion Detection in 5G Ecosystem Using Optimized BERT-Based Model," in *Proceedings of the 2024 20th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)* (Paris, 2024), 62–67, https://doi.org/10.1109/WiMob61911.2024.10770340.

105. N. Ziems, G. Liu, J. Flanagan, and M. Jiang, "Explaining Tree Model Decisions in Natural Language for Network Intrusion Detection," (2023), https://arxiv.org/abs/2310.19658.

106. Y. A. Farrukh, S. Wali, I. Khan, and N. D. Bastian, "XG-NID: Dual-Modality Network Intrusion Detection Using a Heterogeneous Graph Neural Network and Large Language Model," *Expert Systems With Applications* 287 (2025): 128089, https://doi.org/10.1016/j.eswa.2025.128089.

107. T. Ali and P. Kostakos, "HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI With Large Language Models (LLMs)," (2023), https://arxiv.org/abs/2309.16021.

108. M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A Survey of Network-Based Intrusion Detection Data Sets," *Computers & Security* 86 (2019): 147–167, https://doi.org/10.1016/j.cose.2019.06.005.

109. A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems," *IEEE Access* 8 (2020): 165130–165150, https://doi.org/10.1109/ACCESS.2020.3022862.

110. KnowBe4, "Phishing Threat Trends Report 5," (2025), https://www.knowbe4.com.

111. K. Misra and J. T. Rayz, "LMs go Phishing: Adapting Pre-Trained Language Models to Detect Phishing Emails," in *Proceedings of the 2022 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT)* (Niagara Falls, 2022), 135–142, https://doi.org/10.1109/WI-IAT55865.2022.00028.

112. N. Abdullahalam, "Phishing Email Dataset," (2024), https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset.

113. A. Das, A. El Aassal, S. Baki, and R. M. Verma, "First Security & Privacy Analytics Anti-Phishing Shared Task (IWSPA-AP 2018)," (2018), https://dasavisha.github.io/IWSPA-sharedtask/.

114. Y. Wang, W. Zhu, H. Xu, Z. Qin, K. Ren, and W. Ma, "A Large-Scale Pretrained Deep Model for Phishing URL Detection," in *Proceedings of the ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (Rhodes Island, 2023), 1–5, https://doi.org/10.1109/ICASSP49357.2023.10095719.

115. Common Crawl Foundation, "Common Crawl—Free, Open Web-Crawl Data," (2007), https://commoncrawl.org/.

116. Cisco Talos Intelligence Group, "[dataset] Phishtank: A Community-Driven Phishing Url Clearinghouse," (2006), https://phishtank.org/.

117. F. Heiding, B. Schneier, A. Vishwanath, J. Bernstein, and P. S. Park, "Devising and Detecting Phishing Emails Using Large Language Models," *IEEE Access* 12 (2024): 42131–42146, https://doi.org/10.1109/ACCESS.2024.3375882.

118. R. Chataut, P. K. Gyawali, and Y. Usman, "Can AI Keep You Safe? A Study of Large Language Models for Phishing Detection," in *Proceedings of the 2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)* (Las Vegas, NV, 2024), 0548–0554, https://doi.org/10.1109/CCWC60891.2024.10427626.

119. subhajournal, "[dataset] Phishing Emails Dataset," (2023), https://www.kaggle.com/datasets/subhajournal/phishingemails.

120. F. Trad and A. Chehab, "Prompt Engineering or Fine-Tuning? A Case Study on Phishing Detection With Large Language Models," *Machine Learning and Knowledge Extraction* 6, no. 1 (2024): 367–384, https://doi.org/10.3390/make6010018.

121. A. Hannousse and S. Yahiouche, "Web Page Phishing Detection," *Mendeley Data* 3 (2021).

122. H. Patel, U. Rehman, and F. Iqbal, "Evaluating the Efficacy of Large Language Models in Identifying Phishing Attempts," in *Proceedings of the 2024 16th International Conference on Human System Interaction (HSI)* (Paris, 2024), 1–7, https://doi.org/10.1109/HSI61632.2024.10613528.

123. R. Tatman, "[dataset] Fraudulent E-Mail Corpus," (2016), https://www.kaggle.com/datasets/rtatman/fraudulent-email-corpus.

124. J. Lee, P. Lim, B. Hooi, and D. M. Divakaran, "Multimodal Large Language Models for Phishing Webpage Detection and Identification," in *Proceedings of the 2024 APWG Symposium on Electronic Crime Research (eCrime)* (Boston, MA, 2024), 1–13, https://doi.org/10.1109/eCrime66200.2024.00007.

125. V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczyński, and W. Joosen, "Tranco: A Research-Oriented Top Sites Ranking Hardened against Manipulation," (2018), https://tranco-list.eu/.

126. OpenPhish, "[dataset] Openphish: Real-Time Phishing Intelligence," (2025), https://openphish.com/.

127. Y. Li, C. Huang, S. Deng, et al., "KnowPhish: Large Language Models Meet Multimodal Knowledge Graphs for Enhancing Reference-Based Phishing Detection," in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)* (Philadelphia, PA, 2024), 793–810.

128. APW Group, "[dataset] Anti-Phishing Working Group (APWG)," (2025), https://apwg.org/.

129. H. Wang and B. Hooi, "Automated Phishing Detection Using URLs and Webpages," (2024), https://arxiv.org/abs/2408.01667.

130. T. Koide, H. Nakano, and D. Chiba, "ChatPhishDetector: Detecting Phishing Sites Using Large Language Models," *IEEE Access* 12 (2024): 154381–154400, https://doi.org/10.1109/ACCESS.2024.3483905.

131. F. Trad and A. Chehab, "Large Multimodal Agents for Accurate Phishing Detection With Enhanced Token Optimization and Cost Reduction," in *Proceedings of the 2024 2nd International Conference on Foundation and Large Language Models (FLLM)* (Dubai, 2024), 229–237, https://doi.org/10.1109/FLLM63129.2024.10852444.

132. K. L. Chiew, E. H. Chang, C. L. Tan, J. Abdullah, and K. S. C. Yong, "Building Standard Offline Anti-Phishing Dataset for Benchmarking,"

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

International Journal of Engineering & Technology 7 (2018): 7–14, https://doi.org/10.14419/ijet.v7i4.31.23333.

133. D. Nahmias, G. Engelberg, D. Klein, and A. Shabtai, "Prompted Contextual Vectors for Spear-Phishing Detection," (2024), https://arxiv.org/abs/2402.08309.

134. S. S. Roy and S. Nilizadeh, "PhishLang: A Lightweight, Client-Side Phishing Detection Framework Using Mobilebert for Real-Time, Explainable Threat Mitigation," (2024), https://arxiv.org/html/2408.05667v2.

135. Lindsey98, "[dataset] Phishpedia: A Hybrid Deep Learning Based Approach to Visually Identify Phishing Webpages," (2021), https://github.com/lindsey98/Phishpedia.

136. G. Desolda, F. Greco, and L. Viganò, "Apollo: A GPT-Based Tool to Detect Phishing Emails and Generate Explanations that Warn Users," *Proceedings of the ACM on Human-Computer Interaction* 9, no. 4 (2025): 1–33, https://doi.org/10.1145/3733049.

137. dAS Project, "Spamassassin Public Mail Corpus," (2002), https://spamassassin.apache.org/old/publiccorpus/.

138. O. K. Sahingoz, E. Bube, and E. Kugu, "DEPHIDES: Deep Learning Based Phishing Detection System," *IEEE Access* 12 (2024): 8052–8070, https://doi.org/10.1109/ACCESS.2024.3352629.

139. J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-Agents Enhances Large Language Model Capabilities," (2024), https://arxiv.org/abs/2406.04692.

140. CheedCheed, "[Dataset]top1m: Top 1 Million Websites," (2025), https://www.kaggle.com/datasets/cheedcheed/top1m.

141. G. J. Benston and A. L. Hartgraves, "Enron: What Happened and What We Can Learn From It," *Journal of Accounting and Public Policy* 21, no. 2 (2002): 105–127, https://doi.org/10.1016/S0278-4254(02)00042-X.

142. J. Shetty and J. Adibi, "[Dataset] Enron Email Dataset," (2004), https://www.cs.cmu.edu/enron/.

143. Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights," (2025), https://arxiv.org/abs/2502.07049.

144. Y. Guo, C. Patsakis, Q. Hu, Q. Tang, and F. Casino, "Outside the Comfort Zone: Analysing LLM Capabilities in Software Vulnerability Detection," in *Proceedings of the 2024 European Symposium on Research in Computer Security* (Bydgoszcz, 2024), 271–289, https://doi.org/10.1007/978-3-031-70879-4_14.

145. A. A. Mahyari, "Harnessing the Power of LLMs in Source Code Vulnerability Detection," in *Proceedings of the MILCOM 2024-2024 IEEE Military Communications Conference (MILCOM)* (Washington, DC, 2024), 251–256, https://doi.org/10.1109/MILCOM61039.2024.10774025.

146. B. Steenhoek, M. M. Rahman, M. K. Roy, M. S. Alam, E. T. Barr, and W. Le, "A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection," (2024), https://arxiv.org/html/2403.17218v1.

147. A. Cheshkov, P. Zadorozhny, and R. Levichev, "Evaluation of ChatGPT Model for Vulnerability Detection," (2023), https://arxiv.org/abs/2304.07232.

148. M. Fu, C. K. Tantithamthavorn, V. Nguyen, and T. Le, "ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?" in *Proceedings of the 2023 30th Asia-Pacific Software Engineering Conference (APSEC)* (Seoul, 2023), 632–636, https://doi.org/10.1109/APSEC60848.2023.00085.

149. M. Fu and C. Tantithamthavorn, "LineVul: A Transformer-Based Line-Level Vulnerability Prediction," in *Proceedings of the 19th International Conference on Mining Software Repositories* (Pittsburgh, PA, 2022), 608–620, https://doi.org/10.1145/3524842.3528452.

150. M. Omar and S. Shiaeles, "VulDetect: A Novel Technique for Detecting Software Vulnerabilities Using Language Models," in *Proceedings of the 2023 IEEE International Conference on Cyber Security and Resilience (CSR)* (Venice, 2023), 105–110, https://doi.org/10.1109/CSR57506.2023.10224924.

151. A. Z. Yang, C. Le Goues, R. Martins, and V. Hellendoorn, "Large Language Models for Test-free Fault Localization," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering* (Lisbon, 2024), 1–12, https://doi.org/10.1145/3597503.3623342.

152. A. Mechri, M. A. Ferrag, and M. Debbah, "SecureQwen: Leveraging LLMs for Vulnerability Detection in Python Codebases," *Computers & Security* 148 (2025): 104151, https://doi.org/10.1016/j.cose.2024.104151.

153. M. A. Ferrag, A. Battah, N. Tihanyi, et al., "SecureFalcon: Are We There Yet in Automated Software Vulnerability Detection With LLMs?" *IEEE Transactions on Software Engineering* 51, no. 4 (2025): 1248–1265, https://doi.org/10.1109/TSE.2025.3548168.

154. Z. Feng, D. Guo, D. Tang, et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," (2020), https://arxiv.org/abs/2002.08155.

155. E. Nijkamp, B. Pang, H. Hayashi, et al., "CodeGen: An Open Large Language Model for Code With Multi-Turn Program Synthesis," (2022), https://arxiv.org/abs/2203.13474.

156. Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-Aware Unified Pre-Trained Encoder-Decoder Models for Code Understanding and Generation," (2021), https://arxiv.org/abs/2109.00859.

157. V. Nguyen, S. Nepal, T. Wu, X. Yuan, and C. Rudolph, "SAFE: Advancing Large Language Models in Leveraging Semantic and Syntactic Relationships for Software Vulnerability Detection," (2024), https://arxiv.org/abs/2409.00882.

158. S. Z. Ridoy, M. S. H. Shaon, A. Cuzzocrea, and M. S. Akter, "EnStack: An Ensemble Stacking Framework of Large Language Models for Enhanced Vulnerability Detection in Source Code," in *Proceedings of the 2024 IEEE International Conference on Big Data (BigData)* (Washington, DC, 2024), 6356–6364, https://doi.org/10.1109/BigData62323.2024.10825609.

159. G. Lu, X. Ju, X. Chen, W. Pei, and Z. Cai, "GRACE: Empowering LLM-Based Software Vulnerability Detection With Graph Structure and In-Context Learning," *Journal of Systems and Software* 212 (2024): 112031, https://doi.org/10.1016/j.jss.2024.112031.

160. Y. Yang, X. Zhou, R. Mao, et al., "DLAP: A Deep Learning Augmented Large Language Model Prompting Framework for Software Vulnerability Detection," *Journal of Systems and Software* 219 (2025): 112234, https://doi.org/10.1016/j.jss.2024.112234.

161. H. Ding, Y. Liu, X. Piao, H. Song, and Z. Ji, "SmartGuard: An LLM-Enhanced Framework for Smart Contract Vulnerability Detection," *Expert Systems With Applications* 269 (2025): 126479, https://doi.org/10.1016/j.eswa.2025.126479.

162. W. Wang, E. Liu, X. Guo, X. Hu, I. Grishchenko, and D. Lie, "ANVIL: Anomaly-Based Vulnerability Identification Without Labelled Training Data," (2024), https://arxiv.org/abs/2408.16028.

163. X. Du, G. Zheng, K. Wang, et al., "Vul-RAG: Enhancing LLM-Based Vulnerability Detection via Knowledge-Level Rag," (2024), https://arxiv.org/abs/2406.11147.

164. Z. Sheng, F. Wu, X. Zuo, C. Li, Y. Qiao, and L. Hang, "LProtector: An LLM-Driven Vulnerability Detection System," (2024), https://arxiv.org/abs/2411.06493.

165. Z. Mao, J. Li, D. Jin, M. Li, and K. Tei, "Multi-Role Consensus Through LLMs Discussions for Vulnerability Detection," in *Proceedings of the 2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)* (Cambridge, 2024), 1318–1319, https://doi.org/10.1109/QRS-C63300.2024.00173.

166. Z. Wei, J. Sun, Y. Sun, et al., "Advanced Smart Contract Vulnerability Detection via Llm-Powered Multi-Agent Systems," *IEEE*

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Transactions on Software Engineering 51, no. 10 (2025): 2830–2846, https://doi.org/10.1109/TSE.2025.3597319.

167. J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ Code Vulnerability Dataset With Code Changes and CVE Summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories* (Seoul, 2020), 508–512, https://doi.org/10.1145/3379597.3387501.

168. NIST Software Assurance Metrics and Tool Evaluation, "[Dataset] Software Assurance Reference Dataset," (2006), https://samate.nist.gov/SARD.

169. Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities," *IEEE Transactions on Dependable and Secure Computing* 19, no. 4 (2021): 2244–2258, https://doi.org/10.1109/TDSC.2021.3051525.

170. Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks," *Advances in Neural Information Processing Systems* 32 (2019).

171. S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep Learning Based Vulnerability Detection: Are We There Yet?" *IEEE Transactions on Software Engineering* 48, no. 9 (2021): 3280–3296, https://doi.org/10.1109/TSE.2021.3087402.

172. Y. Zheng, S. Pujar, B. Lewis, et al., "D2A: A Dataset Built for AI-Based Vulnerability Detection Methods Using Differential Analysis," in *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (Madrid, 2021), 111–120, https://doi.org/10.1109/ICSE-SEIP52600.2021.00020.

173. J. Haurogné, N. Basheer, and S. Islam, "Vulnerability Detection Using BERT Based LLM Model With Transparency Obligation Practice Towards Trustworthy AI," *Machine Learning with Applications* 18 (2024): 100598, https://doi.org/10.1016/j.mlwa.2024.100598.

174. Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses* (Hong Kong, 2023), 654–668, https://doi.org/10.1145/3607199.3607242.

175. R. Russell, L. Kim, L. Hamilton, et al., "Automated Vulnerability Detection in Source Code Using Deep Representation Learning," in *Proceedings of the 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)* (Orlando, FL, 2018), 757–762, https://doi.org/10.1109/ICMLA.2018.00120.

176. Q. Mao, Z. Li, X. Hu, K. Liu, X. Xia, and J. Sun, "Towards Effectively Detecting and Explaining Vulnerabilities Using Large Language Models," (2024), https://arxiv.org/html/2406.09701v2.

177. N. Tihanyi, T. Bisztray, R. Jain, M. A. Ferrag, L. C. Cordeiro, and V. Mavroeidis, "The Formai Dataset: Generative AI in Software Security through the Lens of Formal Verification," in *Proceedings of the 19th International Conference on Predictive Models and Data Analytics in Software Engineering* (San Francisco, CA, 2023), 33–43, https://doi.org/10.1145/3617555.3617874.

178. A. Ghaleb and K. Pattabiraman, "How Effective Are Smart Contract Analysis Tools? Evaluating Smart Contract Static Analysis Tools Using Bug Injection," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Los Angeles, 2020), 415–427, https://doi.org/10.1145/3395363.3397385.

179. A. Hazimeh, A. Herrera, and M. Payer, "Magma: A Ground-Truth Fuzzing Benchmark," *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, no. 3 (2020): 1–29, https://doi.org/10.1145/3428334.

180. Y. Ding, Y. Fu, O. Ibrahim, et al., "Vulnerability Detection With Code Language Models: How Far Are We?" (2024), https://arxiv.org/abs/2403.18624.

181. L. Wartschinski, Y. Noller, T. Vogel, T. Kehrer, and L. Grunske, "VUDENC: Vulnerability Detection With Deep Learning on a Natural Codebase for Python," *Information and Software Technology* 144 (2022): 106809, https://doi.org/10.1016/j.infsof.2021.106809.

182. G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: Automated Collection of Vulnerabilities and Their Fixes From Open-Source Software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering* (Madrid, Spain, 2021), 30–39, https://doi.org/10.5281/zenodo.5111494.

183. A. Rahali and M. A. Akhloufi, "MalBERT: Malware Detection Using Bidirectional Encoder Representations From Transformers," in *Proceedings of the 2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)* (Melbourne, 2021), 3226–3231, https://doi.org/10.1109/SMC52423.2021.9659287.

184. Y. E. Seyyar, A. G. Yavuz, and H. M. Ünver, "An Attack Detection Framework Based on BERT and Deep Learning," *IEEE Access* 10 (2022): 68633–68644, https://doi.org/10.1109/ACCESS.2022.3185748.

185. T. Vörös, S. P. Bergeron, and K. Berlin, "Web Content Filtering Through Knowledge Distillation of Large Language Models," in *Proceedings of the 2023 IEEE/WIC International Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT)* (Venice, 2023), 357–361, https://doi.org/10.1109/WI-IAT59888.2023.00058.

186. J. Liu, Y. Zhao, Y. Feng, Y. Hu, and X. Ma, "SeMalBERT: Semantic-Based Malware Detection With Bidirectional Encoder Representations From Transformers," *Journal of Information Security and Applications* 80 (2024): 103690, https://doi.org/10.1016/j.jisa.2023.103690.

187. S. Jamal, H. Wimmer, and I. H. Sarker, "An Improved Transformer-Based Model for Detecting Phishing, Spam and Ham Emails: A Large Language Model Approach," *Security and Privacy* 7, no. 5 (2024): e402, https://doi.org/10.1002/spy2.402.

188. D. Du, X. Guan, Y. Liu, et al., "MAD-LLM: A Novel Approach for Alert-Based Multi-Stage Attack Detection via LLM," in *Proceedings of the 2024 IEEE International Symposium on Parallel and Distributed Processing With Applications (ISPA)* (Kaifeng, 2024), 2046–2053, https://doi.org/10.1109/ISPA63168.2024.00279.

189. S. Si, Y. Wu, L. Tang, Y. Zhang, J. Wosik, and Q. Su, "Evaluating the Performance of Chatgpt for Spam Email Detection," (2024), https://arxiv.org/abs/2402.15537.

190. M. A. Uddin, M. N. Islam, L. Maglaras, H. Janicke, and I. H. Sarker, "ExplainableDetector: Exploring Transformer-Based Language Modeling Approach for SMS Spam Detection With Explainability Analysis," *Digital Communications and Networks* 11, no. 5 (2025): 1504–1518, https://doi.org/10.1016/j.dcan.2025.07.008.

191. P. Ranade, A. Piplai, A. Joshi, and T. Finin, "CyBERT: Contextualized Embeddings for the Cybersecurity Domain," in *Proceedings of the 2021 IEEE International Conference on Big Data (Big Data)* (Orlando, FL, 2021), 3334–3342, https://doi.org/10.1109/BigData52589.2021.9671824.

192. Y. Park and W. You, "A Pretrained Language Model for Cyber Threat Intelligence," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track* (Singapore, 2023), 113–122, https://doi.org/10.18653/v1/2023.emnlp-industry.12.

193. M. Bayer, P. Kuehn, R. Shanehsaz, and C. Reuter, "CySecBERT: A Domain-Adapted Language Model for the Cybersecurity Domain," *ACM Transactions on Privacy and Security* 27, no. 2 (2024): 1–20, https://doi.org/10.1145/3652594.

194. N. Rani, B. Saha, V. Maurya, and S. K. Shukla, "TTPHunter: Automated Extraction of Actionable Intelligence as TTPs From Narrative Threat Reports," in *Proceedings of the 2023 Australasian Computer Science Week* (Melbourne, 2023), 126–134, https://doi.org/10.1145/3579375.3579391.

195. E. Aghaei and E. Al-Shaer, "CVE-Driven Attack Technique Prediction With Semantic Information Extraction and a Domain-Specific Language Model," (2023), https://arxiv.org/abs/2309.02785.

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

196. E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer, "SecureBERT: A Domain-Specific Language Model for Cybersecurity," in *Proceedings of the 2022 International Conference on Security and Privacy in Communication Systems* (Springer, 2022), 39–56, https://doi.org/10.1007/978-3-031-25538-0_3.

197. Gartner Inc, "Definition: Threat Intelligence," (2013), https://www.gartner.com/en/documents/2487216.

198. Cisco Talos Intelligence Group, "Cisco Talos: Comprehensive Threat Intelligence and Research," (2025), https://www.talosintelligence.com/.

199. Unit 42, Palo Alto Networks, "Unit 42: Cyber Threat Intelligence and Incident Response," (2025), https://unit42.paloaltonetworks.com/.

200. The Hacker News, "The Hacker News – Cybersecurity News," (2025), https://thehackernews.com/.

201. Bleeping Computer, "Bleeping Computer – Cybersecurity News and Support," (2025), https://www.bleepingcomputer.com/.

202. Krebs on Security, "Krebs on Security – Cybersecurity News and Investigations," (2025), https://krebsonsecurity.com/.

203. Schneier on Security, "Schneier on Security – Security News and Commentary," (2025), https://www.schneier.com/.

204. Y. You, J. Jiang, Z. Jiang, et al., "TIM: Threat Context-Enhanced TTP Intelligence Mining on Unstructured Threat Data," *Cybersecurity* 5, no. 1 (2022): 3, https://doi.org/10.1186/s42400-021-00106-5.

205. J. Liu, J. Yan, J. Jiang, et al., "TriCTI: An Actionable Cyber Threat Intelligence Discovery System via Trigger-Enhanced Neural Network," *Cybersecurity* 5, no. 1 (2022): 8, https://doi.org/10.1186/s42400-022-00110-3.

206. H. Jo, Y. Lee, and S. Shin, "Vulcan: Automatic Extraction and Analysis of Cyber Threat Intelligence From Unstructured Text," *Computers & Security* 120 (2022): 102763, https://doi.org/10.1016/j.cose.2022.102763.

207. M. T. Alam, D. Bhusal, Y. Park, and N. Rastogi, "Looking Beyond IoCs: Automatically Extracting Attack Patterns From External CTI," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses* (Hong Kong, 2023), 92–108, https://doi.org/10.1145/3607199.3607208.

208. G. Siracusano, D. Sanvito, R. Gonzalez, et al., "Time for aCTIon: Automated Analysis of Cyber Threat Intelligence in the Wild," (2023), https://arxiv.org/abs/2307.10214.

209. Y. Zhang, T. Du, Y. Ma, et al., "AttacKG+: Boosting Attack Knowledge Graph Construction With Large Language Models," (2024), https://arxiv.org/abs/2405.04753.

210. Y. Hu, F. Zou, J. Han, X. Sun, and Y. Wang, "LLM-TIKG: Threat Intelligence Knowledge Graph Construction Utilizing Large Language Model," *Computers & Security* 145 (2024): 103999, https://doi.org/10.1016/j.cose.2024.103999.

211. N. Rani, B. Saha, V. Maurya, and S. K. Shukla, "TTPXHunter: Actionable Threat Intelligence Extraction as TTPs From Finished Cyber Threat Reports," *Digital Threats: Research and Practice* 5, no. 4 (2024): 1–19, https://doi.org/10.1145/3696427.

212. Y. Schwartz, L. Ben-Shimol, D. Mimran, Y. Elovici, and A. Shabtai, "LLMCloudHunter: Harnessing LLMs for Automated Extraction of Detection Rules From Cloud-Based CTI," in *Proceedings of the ACM on Web Conference 2025* (Sydney, 2025), 1922–1941, https://doi.org/10.1145/3696410.3714798.

213. Y. Cheng, O. Bajaber, S. A. Tsegai, D. Song, and P. Gao, "CTINexus: Leveraging Optimized LLM In-Context Learning for Constructing Cybersecurity Knowledge Graphs under Data Scarcity," (2024), https://arxiv.org/html/2410.21060v1.

214. F. Perrina, F. Marchiori, M. Conti, and N. V. Verde, "AGIR: Automating Cyber Threat Intelligence Reporting With Natural Language Generation," in *Proceedings of the 2023 IEEE International Conference on Big Data (BigData)* (Sorrento, 2023), 3053–3062, https://doi.org/10.1109/BigData59044.2023.10386116.

215. G. Michelet and F. Breitinger, "ChatGPT, Llama, Can You Write My Report? An Experiment on Assisted Digital Forensics Reports Written Using (Local) Large Language Models," *Forensic Science International: Digital Investigation* 48 (2024): 301683, https://doi.org/10.1016/j.fsidi.2023.301683.

216. Threatpost, "Threatpost: The First Stop for Security News," (2026), https://threatpost.com/.

217. The Hacker News, "The Hacker News: #1 Trusted Cybersecurity News Platform," (2026), https://thehackernews.com/.

218. E. Hemberg, J. Kelly, M. Shlapentokh-Rothman, et al., "Linking Threat Tactics, Techniques, and Patterns With Defensive Weaknesses, Vulnerabilities and Affected Platform Configurations for Cyber Hunting," (2020), https://arxiv.org/abs/2010.00533.

219. S. Ghimire, Y.-Z. Lin, M. Mamun, et al., "HWREx: AI-Enabled Hardware Weakness and Risk Exploration and Storytelling Framework With LLM-Assisted Mitigation Suggestion," *ACM Transactions on Design Automation of Electronic Systems* 30, no. 6 (2025): 1–33, https://doi.org/10.1145/3737459.

220. B. Ampel, S. Samtani, S. Ullman, and H. Chen, "Linking Common Vulnerabilities and Exposures to the Mitre ATT&CK Framework: A Self-Distillation Approach," (2021), https://arxiv.org/abs/2108.01696.

221. S. S. Das, E. Serra, M. Halappanavar, A. Pothen, and E. Al-Shaer, "V2W-BERT: A Framework for Effective Hierarchical Multiclass Classification of Software Vulnerabilities," in *Proceedings of the 2021 IEEE 8th International Conference on Data Science and Advanced Analytics (DSAA)* (Porto, 2021), 1–12, https://doi.org/10.1109/DSAA53316.2021.9564227.

222. S. S. Das, A. Dutta, S. Purohit, E. Serra, M. Halappanavar, and A. Pothen, "Towards Automatic Mapping of Vulnerabilities to Attack Patterns Using Large Language Models," in *Proceedings of the 2022 IEEE International Symposium on Technologies for Homeland Security (HST)* (Boston, MA: IEEE, 2022), 1–7, https://doi.org/10.1109/HST56032.2022.10025459.

223. O. Grigorescu, A. Nica, M. Dascalu, and R. Rughinis, "CVE2ATT&CK: BERT-Based Mapping of CVEs to MITRE ATT&CK Techniques," *Algorithms* 15, no. 9 (2022): 314, https://doi.org/10.3390/a15090314.

224. J. Jin, B. Tang, M. Ma, et al., "Crimson: Empowering Strategic Reasoning in Cybersecurity Through Large Language Models," (2024), https://arxiv.org/abs/2403.00878.

225. B. K. Webb, S. Purohit, and R. Meyur, "Cyber Knowledge Completion Using Large Language Models," (2024), https://arxiv.org/abs/2409.16176.

226. R. Fayyazi, R. Taghdimi, and S. J. Yang, "Advancing Ttp Analysis: Harnessing the Power of Large Language Models With Retrieval Augmented Generation," in *Proceedings of the 2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)* (Honolulu, HI, 2024), 255–261, https://doi.org/10.1109/ACSACW65225.2024.00036.

227. N. Daniel, F. K. Kaiser, S. Giladi, et al., "Labeling Network Intrusion Detection System (Nids) Rules With MITRE ATT&ck Techniques: Machine Learning vs. Large Language Models," *Big Data and Cognitive Computing* 9, no. 2 (2025): 23, https://doi.org/10.3390/bdcc9020023.

228. M. F. Franco, B. Rodrigues, E. J. Scheid, et al., "SecBot: A Business-Driven Conversational Agent for Cybersecurity Planning and Management," in *Proceedings of the 2020 16th International Conference on Network and Service Management (CNSM)* (Izmir, 2020), 1–7, https://doi.org/10.23919/CNSM50824.2020.9269037.

229. T. Singla, D. Anandayuvaraj, K. G. Kalu, T. R. Schorlemmer, and J. C. Davis, "An Empirical Study on Using Large Language Models to

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Analyze Software Supply Chain Security Failures," in *Proceedings of the 2023 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses* (Copenhagen, 2023), 5–15, https://doi.org/10.1145/3605770.3625214.

230. S. Shafee, A. Bessani, and P. M. Ferreira, "Evaluation of LLM-Based Chatbots for OSINT-Based Cyber Threat Awareness," *Expert Systems With Applications* 261 (2025): 125509, https://doi.org/10.1016/j.eswa.2024.125509.

231. M. Kaheh, D. K. Kholgh, and P. Kostakos, "Cyber Sentinel: Exploring Conversational Agents in Streamlining Security Tasks With GPT-4," (2023), https://arxiv.org/abs/2309.16422.

232. S. Rajapaksha, R. Rani, and E. Karafili, "A RAG-Based Question-Answering Solution for Cyber-Attack Investigation and Attribution," in *Proceedings of the 2024 European Symposium on Research in Computer Security* (Bydgoszcz, 2024), 238–256, https://doi.org/10.1007/978-3-031-82362-6_15.

233. H. Ji, J. Yang, L. Chai, et al., "SEvenLLM: Benchmarking, Eliciting, and Enhancing Abilities of Large Language Models in Cyber Threat Intelligence," (2024), https://arxiv.org/abs/2405.03446.

234. C. Paduraru, C. Patilea, and A. Stefanescu, "CyberGuardian: An Interactive Assistant for Cybersecurity Specialists Using Large Language Models," in *Proceedings of the 19th International Conference on Software Technologies (ICSOFT 2024)* (Dijon, 2024), 8–10.

235. V. B. Krishna, "AttackQA: Development and Adoption of a Dataset for Assisting Cybersecurity Operations Using Fine-Tuned and Open-Source LLMs," (2024), https://arxiv.org/abs/2411.01073.

236. D. R. Arikkat, M. Abhinav, N. Binu, et al., "IntellBot: Retrieval Augmented LLM Chatbot for Cyber Threat Knowledge Delivery," in *Proceedings of the 2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN)* (Indore, 2024), 644–651, https://doi.org/10.1109/CICN63059.2024.10847404.

237. S. Mitra, S. Neupane, T. Chakraborty, et al., "LOCALINTEL: Generating Organizational Threat Intelligence From Global and Local Cyber Knowledge," (2024), https://arxiv.org/abs/2401.10036.

238. M. Levi, Y. Allouche, D. Ohayon, and A. Puzanov, "CyberPal.AI: Empowering LLMs with Expert-Driven Cybersecurity Instructions," *Proceedings of the AAAI Conference on Artificial Intelligence* 39, no. 23 (2025): 24402–24412, https://doi.org/10.1609/aaai.v39i23.34618.

239. S. Chopra, H. Ahmad, D. Goel, and C. Szabo, "ChatNVD: Advancing Cybersecurity Vulnerability Assessment With Large Language Models," (2024), https://arxiv.org/abs/2412.04756.

240. S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (St. Julians: Association for Computational Linguistics, 2024), 150–158, https://doi.org/10.18653/v1/2024.eacl-demo.16.

241. N. Tihanyi, M. A. Ferrag, R. Jain, T. Bisztray, and M. Debbah, "CyberMetric: A Benchmark Dataset Based on Retrieval-Augmented Generation for Evaluating LLMs in Cybersecurity Knowledge," in *Proceedings of the 2024 IEEE International Conference on Cyber Security and Resilience (CSR)* (London, 2024), 296–302, https://doi.org/10.1109/CSR61664.2024.10679494.

242. Y. Zhu, W. Meng, Y. Liu, et al., "UniLog: Deploy One Model and Specialize It for All Log Analysis Tasks," (2021), https://arxiv.org/abs/2112.03159.

243. M. Stewart, M. Hodkiewicz, and S. Li, "Large Language Models for Failure Mode Classification: An Investigation," (2023), https://arxiv.org/abs/2309.08181.

244. Y. Chen, H. Xie, M. Ma, et al., "Automatic Root Cause Analysis via Large Language Models for Cloud Incidents," in *Proceedings of the 19th European Conference on Computer Systems* (Athens, 2024), 674–688, https://doi.org/10.1145/3627703.3629553.

245. D. Roy, X. Zhang, R. Bhave, et al., "Exploring LLM-Based Agents for Root Cause Analysis," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, 2024), 208–219, https://doi.org/10.1145/3663529.3663841.

246. X. Zhang, S. Ghosh, C. Bansal, et al., "Automated Root Causing of Cloud Incidents Using In-Context Learning With GPT-4," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, 2024), 266–277, https://doi.org/10.1145/3663529.3663846.

247. P. Mudgal, B. Arbab, and S. S. Kumar, "CrashEventLLM: Predicting System Crashes With Large Language Models," in *Proceedings of the 2024 International Conference on Information Technology and Computing (ICITCOM)* (Yogyakarta, 2024), 72–76, https://doi.org/10.1109/ICITCOM62788.2024.10762255.

248. S. Shan, Y. Huo, Y. Su, Y. Li, D. Li, and Z. Zheng, "Face It Yourselves: An LLM-Based Two-Stage Strategy to Localize Configuration Errors via Logs," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, 2024), 13–25, https://doi.org/10.1145/3650212.3652106.

249. Z. Wang, Z. Liu, Y. Zhang, et al., "RCAgent: Cloud Root Cause Analysis by Autonomous Agents With Tool-Augmented Large Language Models," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management* (Boise, ID, 2024), 4966–4974, https://doi.org/10.1145/3627673.3680016.

250. J. Huang, Z. Jiang, J. Liu, et al., "Demystifying and Extracting Fault-Indicating Information From Logs for Failure Diagnosis," in *Proceedings of the 2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)* (Tsukuba, 2024), 511–522, https://doi.org/10.1109/ISSRE62328.2024.00055.

251. Z. Wang, J. Li, M. Ma, et al., "Large Language Models Can Provide Accurate and Interpretable Incident Triage," in *Proceedings of the 2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)* (Tsukuba, 2024), 523–534, https://doi.org/10.1109/ISSRE62328.2024.00056.

252. L. Ma, W. Yang, Y. Li, et al., "AdaptiveLog: An Adaptive Log Analysis Framework With the Collaboration of Large and Small Language Model," (2025), https://arxiv.org/abs/2501.11031.

253. L. Zhang, T. Jia, M. Jia, Y. Wu, H. Liu, and Y. Li, "ScalaLog: Scalable Log-Based Failure Diagnosis Using LLM," in *Proceedings of the ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (Hyderabad, 2025), 1–5, https://doi.org/10.1109/ICASSP49660.2025.10888670.

254. C. Pei, Z. Wang, F. Liu, et al., "Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis," in *Companion Proceedings of the ACM on Web Conference 2025* (Sydney, 2025), 422–431, https://doi.org/10.1145/3701716.3715225.

255. P. Iyenghar, C. Zimmer, and C. Gregorio, "A Feasibility Study on Chain-of-Thought Prompting for LLM-Based OT Cybersecurity Risk Assessment," in *Proceedings of the 2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)* (Emden, 2025), 1–4, https://doi.org/10.1109/ICPS65515.2025.11087903.

256. J. Yin, M. Tang, J. Cao, and H. Wang, "Apply Transfer Learning to Cybersecurity: Predicting Exploitability of Vulnerabilities by Description," *Knowledge-Based Systems* 210 (2020): 106529, https://doi.org/10.1016/j.knosys.2020.106529.

257. E. Aghaei, E. Al-Shaer, W. Shadid, and X. Niu, "Automated CVE Analysis for Threat Prioritization and Impact Prediction," (2023), https://arxiv.org/abs/2309.03040.

258. Z. Chen, S. Kommrusch, and M. Monperrus, "Neural Transfer Learning for Repairing Security Vulnerabilities in C Code," *IEEE*

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Transactions on Software Engineering 49, no. 1 (2022): 147–165, https://doi.org/10.1109/TSE.2022.3147265.

259. N. T. Islam, J. Khoury, A. Seong, et al., "LLM-Powered Code Vulnerability Repair With Reinforcement Learning and Semantic Reward," (2024), https://arxiv.org/abs/2401.03374.

260. Q. Zhang, C. Fang, B. Yu, W. Sun, T. Zhang, and Z. Chen, "Pre-Trained Model-Based Automated Software Vulnerability Repair: How Far Are We?" *IEEE Transactions on Dependable and Secure Computing* 21, no. 4 (2023): 2507–2525.

261. Y. Wu, N. Jiang, H. V. Pham, et al., "How Effective Are Neural Networks for Fixing Security Vulnerabilities," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, 2023), 1282–1294, https://doi.org/10.1145/3597926.3598135.

262. A. Zibaeirad and M. Vieira, "VulnLLMEval: A Framework for Evaluating Large Language Models in Software Vulnerability Detection and Patching," (2024), https://arxiv.org/abs/2409.10756.

263. A. Khan, G. Liu, and X. Gao, "Code Vulnerability Repair With Large Language Model Using Context-Aware Prompt Tuning," in *Proceedings of the 2025 IEEE Security and Privacy Workshops (SPW)* (San Francisco, CA, 2025), 283–287, https://doi.org/10.1109/SPW67851.2025.00040.

264. M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, "VulRepair: A T5-Based Automated Software Vulnerability Repair," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, 2022), 935–947, https://doi.org/10.1145/3540250.3549098.

265. X. Zhou, K. Kim, B. Xu, D. Han, and D. Lo, "Out of Sight, Out of Mind: Better Automatic Vulnerability Repair by Broadening Input Ranges and Sources," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)* (Lisbon, 2024), https://doi.org/10.1145/3597503.3639222.

266. D. de Fitero-Dominguez, E. Garcia-Lopez, A. Garcia-Cabot, and J.-J. Martinez-Herraiz, "Enhanced Automated Code Vulnerability Repair Using Large Language Models," *Engineering Applications of Artificial Intelligence* 138 (2024): 109291, https://doi.org/10.1016/j.engappai.2024.109291.

267. A. Bensaoud and J. Kalita, "Advancing Software Security: DCode-BERT for Automatic Vulnerability Detection and Repair," *Journal of Industrial Information Integration* 45 (2025): 100834, https://doi.org/10.1016/j.jii.2025.100834.

268. J. Hou, J. Han, C. Huang, N. Wang, and L. Li, "LineJLocRepair: A Line-Level Method for Automated Vulnerability Repair Based on Joint Training," *Future Generation Computer Systems* 166 (2025): 107671, https://doi.org/10.1016/j.future.2024.107671.

269. J. Zhang, C. Wang, A. Li, W. Wang, T. Li, and Y. Liu, "VulAdvisor: Natural Language Suggestion Generation for Software Vulnerability Repair," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, 2024), 1932–1944.

270. R. Fayyazi, S. H. Trueba, M. Zuzak, and S. J. Yang, "ProveRAG: Provenance-Driven Vulnerability Analysis With Automated Retrieval-Augmented LLMs," (2024), https://arxiv.org/html/2410.17406v2.

271. C. Wang, J. Zhang, J. Gao, L. Xia, Z. Guan, and Z. Chen, "ContractTinker: LLM-Empowered Vulnerability Repair for Real-World Smart Contracts," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, 2024), 2350–2353, https://doi.org/10.1145/3691620.3695349.

272. K. An, F. Yang, L. Li, et al., "Nissist: An Incident Mitigation Copilot Based on Troubleshooting Guides," (2024), https://arxiv.org/abs/2402.17531.

273. Z. Liu, "AutoBnB: Multi-Agent Incident Response With Large Language Models," in *Proceedings of the 2025 13th International Symposium on Digital Forensics and Security (ISDFS)* (Boston, MA, 2025), 1–6, https://doi.org/10.1109/ISDFS65363.2025.11012055.

274. S. Baral, S. Saha, and A. Haque, "Autonomous Cyber Incident Response Using Reasoning and Action," in *Proceedings of the 2025 International Wireless Communications and Mobile Computing (IWCMC)* (Abu Dhabi, 2025), 1392–1397, https://doi.org/10.1109/IWCMC65282.2025.11059476.

275. X. Lin, J. Zhang, G. Deng, et al., "IRCopilot: Automated Incident Response With Large Language Models," (2025), https://arxiv.org/abs/2505.20945.

276. S. Hays and J. White, "Employing LLMs for Incident Response Planning and Review," (2024), https://arxiv.org/abs/2403.01271.

277. Y. Ren, J. Wang, Z. Zhao, H. Wen, H. Li, and H. Zhu, "Automated Tactics Planning for Cyber Attack and Defense Based on Large Language Model Agents," *Neural Networks* 191 (2025): 107842, https://doi.org/10.1016/j.neunet.2025.107842.

278. T. Wang, X. Xie, L. Zhang, C. Wang, L. Zhang, and Y. Cui, "ShieldGPT: An LLM-Based Framework for DDoS Mitigation," in *Proceedings of the 8th Asia-Pacific Workshop on Networking* (Sydney, 2024), 108–114, https://doi.org/10.1145/3663408.3663424.

279. J. Wang, G. Chu, J. Wang, et al., "LogExpert: Log-Based Recommended Resolutions Generation Using Large Language Model," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results* (Lisbon, 2024), 42–46, https://doi.org/10.1145/3639476.3639773.

280. R. Zhong, Y. Li, J. Kuang, W. Gu, Y. Huo, and M. R. Lyu, "LogUpdater: Automated Detection and Repair of Specific Defects in Logging Statements," *ACM Transactions on Software Engineering and Methodology* 35, no. 1 (2025): 1–31, https://doi.org/10.1145/3731754.

281. H. S. Galadima, C. Doherty, and R. Brennan, "Towards LLM-Based Synthetic Dataset Generation of Cyber Incident Response Process Logs," in *Proceedings of the 2024 Cyber Research Conference-Ireland (Cyber-RCI)* (Carlow, 2024), 1–4, https://doi.org/10.1109/Cyber-RCI60769.2024.10939563.

282. T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending Root-Cause and Mitigation Steps for Cloud Incidents Using Large Language Models," in *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (Melbourne, 2023), 1737–1749, https://doi.org/10.1109/ICSE48619.2023.00149.

283. S. Akhtar, S. Khan, and S. Parkinson, "LLM-based Event Log Analysis Techniques: A Survey," (2025), https://arxiv.org/abs/2502.00677.

284. L. Kersten, T. Mulders, E. Zambon, C. Snijders, and L. Allodi, "Give Me Structure: Synthesis and Evaluation of a (Network) Threat Analysis Process Supporting Tier 1 Investigations in a Security Operation Center," in *Proceedings of the 19th Symposium on Usable Privacy and Security (SOUPS 2023)* (Anaheim, CA, 2023), 97–111.

285. A. Hermann, *GPT Powered Log Analysis: Enhancing SOC Decision Making for Malicious and Benign Security Log Classification* (University of Twente, 2024).

286. S. El Jaouhari and Y. Etiabi, "FedCTI: Federated Learning and Cyber Threat Intelligence on the Edge for Secure Iot Networks," in *Proceedings of the 13th International Conference on the Internet of Things* (Nagoya, 2023), 98–104, https://doi.org/10.1145/3627050.3627064.

287. G. Lin, S. Qian, and Z. H. Khattak, "FedAV: Federated Learning for Cyberattack Vulnerability and Resilience of Cooperative Driving Automation," *Communications in Transportation Research* 5 (2025): 100175, https://doi.org/10.1016/j.commtr.2025.100175.

288. D. A. Bierbrauer, S. M. Coffey, M. R. Willeke, J. D. Beggs, and N. D. Bastian, "Data-Efficient Federated Learning for Edge Network

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Intrusion Detection," *Engineering Applications of Artificial Intelligence* 150 (2025): 110685, https://doi.org/10.1016/j.engappai.2025.110685.

289. Z. Deng, Y. Guo, C. Han, et al., "AI Agents Under Threat: A Survey of Key Security Challenges and Future Pathways," *ACM Computing Surveys* 57, no. 7 (2025): 1–36, https://doi.org/10.1145/3716628.

290. N. Kshetri, "Transforming Cybersecurity With Agentic AI to Combat Emerging Cyber Threats," *Telecommunications Policy* 49, no. 6 (2025): 102976, https://doi.org/10.1016/j.telpol.2025.102976.

291. S. Tariq, M. Baruwal Chhetri, S. Nepal, and C. Paris, "Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities," *ACM Computing Surveys* 57, no. 9 (2025): 1–38, https://doi.org/10.1145/3723158.

292. A. Habibzadeh, F. Feyzi, and R. E. Atani, "Large Language Models for Security Operations Centers: A Comprehensive Survey," (2025), https://arxiv.org/abs/2509.10858.

1742, 2026, 1, Downloaded from https://onlinelibrary.wiley.com/doi/10.1155/jece/3383674 by Capes, Wiley Online Library on [27/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License